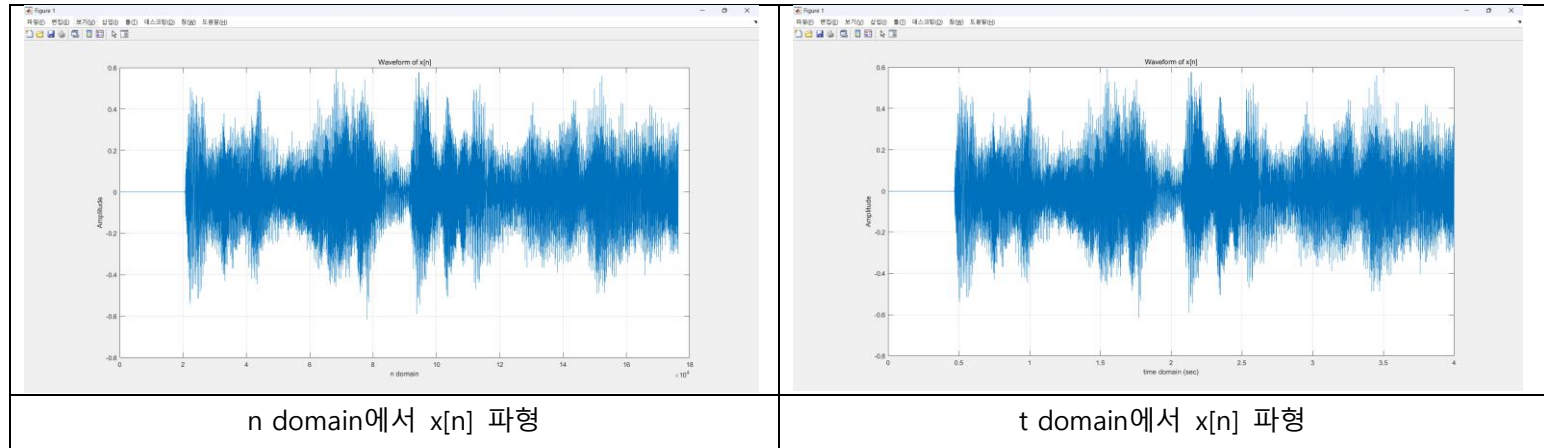


디지털 신호 처리 (EEE4420.01-00) – Simulation2 과제

- 주어진 음향신호 $x[n]$ 을 이용하여 아래 문제들을 순서대로 해결하세요. $x[n]$ 은 파일 'x.wav'을 통해 얻으며, 4초 신호 'x.wav'의 특징은 아래와 같습니다.

'x.wav' : (176,400 point, Sampling rate $f_x = 44,100\text{Hz}$, $T_x = \frac{1}{44,100}\text{sec}$, $\omega_x = 88,200\pi$),

- 1-1. 제공된 음향신호 파일 'x.wav'에서 얻은 $x[n]$ 의 파형을 그리고, 청취하세요.



[x.wav 신호 설명]

Simulation 2의 신호는 4초가량 지속되는 음향 신호이다. 오케스트라의 일부를 잘라서 가져온 음향으로 현악기 및 관악기가 모두 들리며, 이로 인해 음향이 매우 풍부하게 들린다. 즉, 악기를 사용한 음악이고, 악기는 다양한 주파수 대역의 음성을 표현할 수 있으므로, FFT를 취할 경우 저주파부터 고주파 대역을 모두 아우르는 신호일 것으로 추측된다.

[code 구현]

x.wav 파일을 읽고, 신호의 파형을 그린 후, 해당 신호를 소리로 출력하는 작업을 수행한다.

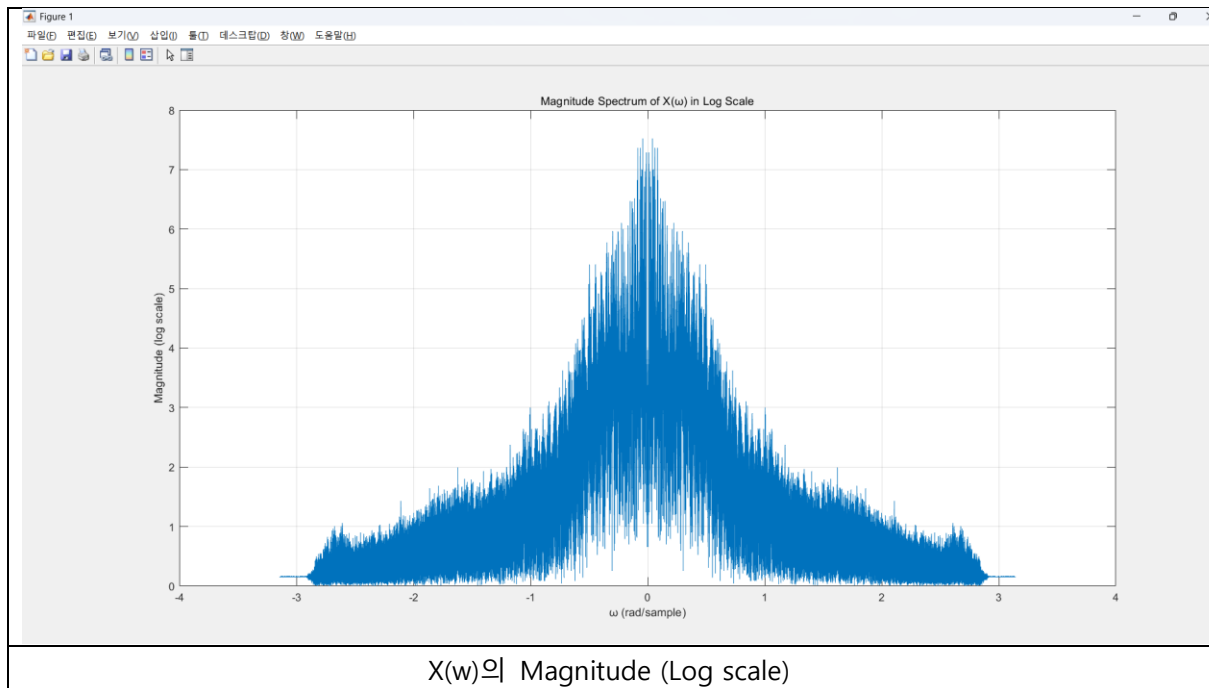
`audioread('x.wav')` 함수를 사용하여 x.wav 파일을 읽고, 그 데이터를 x와 샘플링 주파수 fs로 저장한다. x는 신호 데이터를, fs는 샘플링 주파수를 나타낸다.

n_domain은 신호의 인덱스를 나타내며, t_domain은 신호의 시간 축($1/fs$ 를 곱함)을 계산하여 생성한다. n은 신호의 총 길이를 저장한다.

`plot(n_domain, x)`로 신호의 파형을 그린다. x축은 신호의 time 또는 index값, y축은 신호의 진폭을 나타내며, xlabel, ylabel, title을 사용하여 그래프의 축 이름과 제목을 설정한다.

`sound(x, fs)` 함수를 통해 신호를 실제로 출력하여, 사용자가 해당 신호를 들을 수 있게 한다.

1-2. 신호 $x[n]$ 의 Spectrum을 $X(\omega)$ 라고 했을 때, $X(\omega)$ 의 Magnitude 그래프를 Log scale로 표현



[X(ω)신호 설명]

$x.wav$ 를 FFT하여 ω domain으로 magnitude를 그리고, 크기는 log scale로 표시하였다. $x.wav$ 신호를 들었을 때, 추측했던 것과 같이 오케스트라 연주이므로, 전대역의 주파수에 신호의 정보가 존재하는 것을 볼 수 있다. 물론, 저주파 대역에 가장 많은 정보가 존재하는 것을 확인할 수 있다.

즉, 이 신호는 $-\pi \sim \pi$ 까지 대역에 정보가 모두 존재하는 신호이다.

또한, 저주파 대역에서 가장 큰 magnitude는 log scale로 7~8정도이다. 이는 linear scale로 1800 ~ 2000정도의 크기를 갖는다.

Interpolation 및 Decimation을 하게 되면, 신호의 주파수 축이 변할 뿐만 아니라, 신호의 크기도 변하므로, 크기 및 주파수 대역을 짚고 넘어간다.

[code 구현]

`fft(x)` 함수를 사용하여 신호 x 의 Fourier Transform를 계산한다.

`fftshift(X)`를 통해 결과를 알아보기 쉽도록 FFT의 주파수 성분을 중앙으로 이동시킨다.

주파수 축을 생성한다. `linspace(-pi, pi, n)`를 통해 주파수 범위 $-\pi$ 에서 π 까지 원래 $x[n]$ 의 길이만큼 분할된 축을 생성한다.

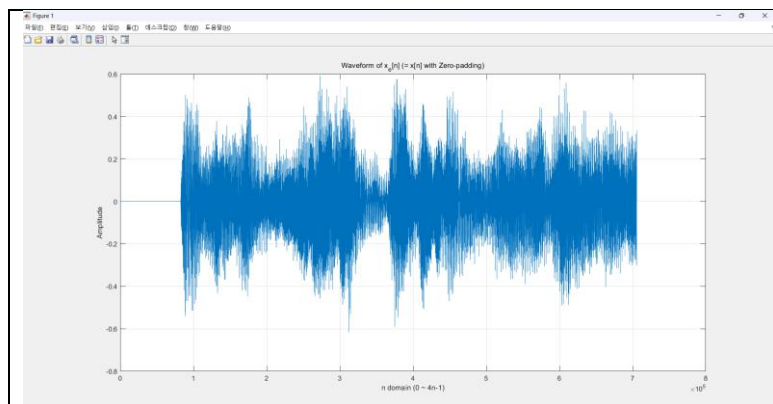
`abs(X_shifted)`를 사용하여 FFT 결과의 Magnitude를 계산하고, 이를 로그 스케일로 변환한다.

`log(1 + X_magnitude)`는 작은 값을 시각적으로 강조하기 위해서 사용한다.

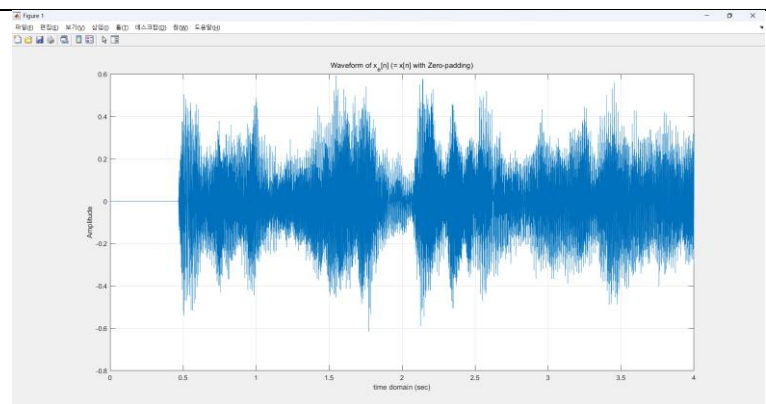
1-3. Interpolation을 하기 전에, 이전 신호 $x[n]$ 사이사이에 0을 삽입하는 과정 (Zero-padding)을 진행합니다. Zero-padding 한 신호를 $x_e[n]$ 라 하면, 신호 $x[n]$ 와 $x_e[n]$ 는 아래와 같은 관계를 갖습니다. ($L=4$)

$$x_e[n] = \begin{cases} x[n/L] & \text{for } n=Lk \text{ where } k=0,1,2,3,\dots \\ 0 & \text{otherwise} \end{cases}$$

1-4. Zero-padding된 신호 $x_e[n]$ 의 파형을 그리고, 신호 $x_e[n]$ 을 청취하세요. 원본 신호 $x[n]$ 와 어떤 차이가 있는지 서술하세요.



n domain에서 $x_e[n]$ 파형



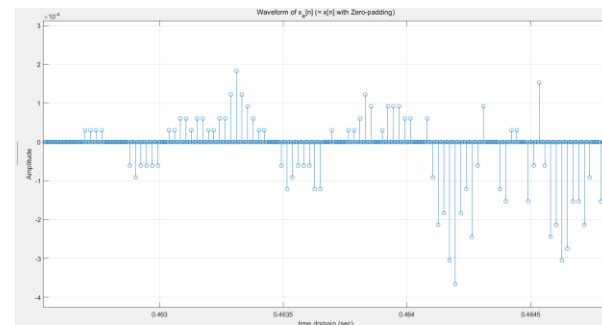
t domain에서 $x_e[n]$ 파형

[$x_e[n]$ 신호 설명]

$x_e[n]$ 은 기존 신호 $x[n]$ 을 4번에 한 번씩 할당해주고, 나머지 3번은 zero padding을 해준 신호이다. 이로 인하여 샘플의 수가 4배가 되었다. 오른쪽 사진을 보면, 원 신호의 샘플과 샘플 사이에 0의 신호가 3개씩 존재하는 것을 확인 가능하다.

[음성 청취 설명]

$x_e[n]$ 은 사실 신호의 정보는 똑같은데, 그저, 샘플의 개수를 늘리기 위해 zero padding이라는 기법을 이용한 것이다. 따라서, 음성을 들을 때, 기존의 신호의 sampling rate의 4배를 사용하여야 음성이 4초동안 제대로 나온다. 더불어, 신호의 세기가 0인 샘플이 3개 원 신호의 샘플이 1개이므로, 실제 소리를 청취하였을 때, 가장 큰 특징은 소리의 세기 약화이다. 그 이유는 샘플의 크기가 0인 부분은 소리가 나지 않을 것이기 때문이다. 또한, 약간 소리가 매끄럽지 못한 느낌이 들었다.



Energy: $E = \sum_{n=0}^{N-1} x[n] ^2$	Power: $P = \frac{1}{N} \sum_{n=0}^{N-1} x[n] ^2$
Energy of $x[n] = E_x = \sum_{n=0}^{N-1} x[n] ^2$	Power of $x[n] = P = \frac{1}{N} \sum_{n=0}^{N-1} x[n] ^2$
Energy of $x_e[n] = E_x = \sum_{n=0}^{4N-1} x_e[n] ^2$	Power of $x_e[n] = \frac{1}{4N} \sum_{n=0}^{4N-1} x_e[n] ^2$

위 표를 보면 $x_e[n]$ 과 $x[n]$ 의 Energy는 동일하지만, Power는 1/4 차이가 나는 것을 확인할 수 있다. 따라서, 수식으로 봤을 때, 음량 차이가 왜 나는지를 수학적으로 알 수 있다.

[code 구현]

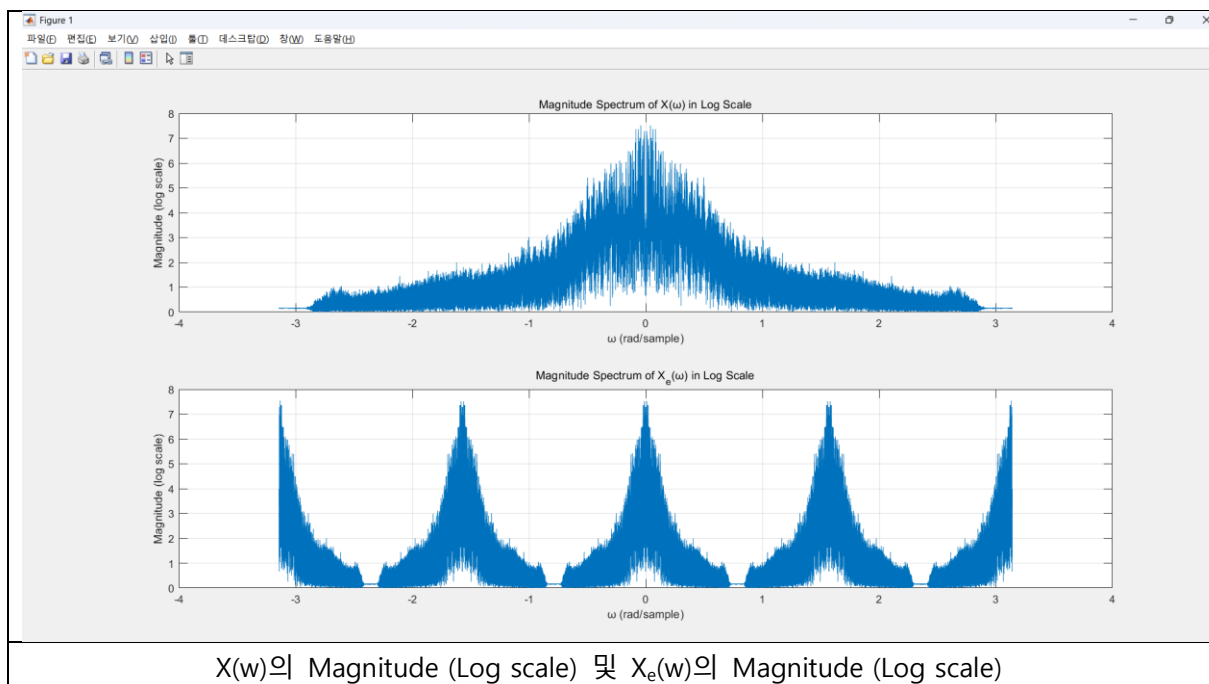
$L = 4$ 는 Interpolation 비율을 설정하는 값으로, 신호의 각 샘플 사이에 3개의 0을 삽입하여 신호의 길이를 4배로 확장한다.

$x_e = \text{zeros}(L*n, 1)$ 로 0으로 채워진 새로운 신호 배열 x_e 를 생성한다. 그런 다음, $x_e(1:L:end) = x$ 로 원래 신호 $x[n]$ 의 값을 x_e 배열의 1, 5, 9, ... 번째 위치에 삽입하여 신호를 확장한다. 이 과정은 신호의 각 샘플 사이에 3개의 0을 추가하는 작업이다.

n_domain_e 는 새로운 신호 x_e 의 인덱스를 나타내며, t_domain_e 는 시간 축을 계산하여 생성한다. 이때 시간 축은 원래 신호의 샘플링 주파수 f_s 와 보간 비율 L 을 고려하여 계산된다.

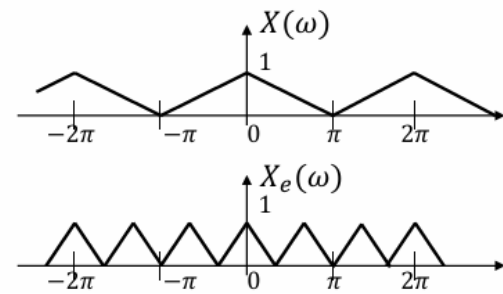
앞으로 나올 $x_e[n]$, $x_{i5}[n]$, $x_{i13}[n]$ 은 모두 upsample된 샘플 수 $L*f_s$ 를 가진다.

1-5. $x_e[n]$ 의 Spectrum을 $X_e(\omega)$ 라고 했을 때, $X_e(\omega)$ 의 Magnitude 그래프를 log scale로 그리세요. 또 Spectrum $X(\omega)$ 와 비교하세요.



[$X_e(\omega)$ 설명 및 $X(\omega)$ 와 비교]

앞서 언급했듯이 Interpolation 및 Decimation을 하게 되면, 신호의 주파수 축이 변할 뿐만 아니라, 신호의 크기도 변하므로, 크기 및 주파수 대역을 짚고 넘어가야 한다.



1) 신호의 크기: $X_e(\omega)$ 와 $X(\omega)$ 모두 크기는 동일하다.

2) 주파수 대역: $X_e(\omega)$ 의 경우 $-\pi \sim \pi$ 까지 동일한 Frequency Response가 4번 나온다. 즉, 4개 중 1개만 봤을 때, $-\pi/4 \sim \pi/4$ 까지의 대역을 차지한다고 볼 수 있다.

3) 그 외: $X_e(\omega)$ 의 경우 4개의 같은 Frequency Response가 반복되므로, 사실상 의미 없는 정보의 반복이다. 즉, 정보량이 많아진 것이 아니다.

신호의 크기는 똑같지만, 의미 있는 정보만 봤을 때, 주파수 대역은 4배가량 차이 난다고 볼 수 있다. 따라서, Parseval 정리에 따라, $-\pi \sim \pi$ 까지의 energy는 같을 수 있어도, 의미 있는 정보만을 본다면, $X(\omega)$ 는 $-\pi \sim \pi$ 까지 $X_e(\omega)$ 는 $-\pi/4 \sim \pi/4$ 까지만을 보기에, 신호의 power가 4배 차이 나는 것으로 볼 수 있다.

[수식 유도]

$$x_e[n] = \sum_{k=-\infty}^{\infty} x[k] \delta[n - kL]$$

$$X_e(\omega) = \sum_{n=-\infty}^{\infty} \sum_{k=-\infty}^{\infty} x[k] \delta[n - kL] e^{-j\omega n} = \sum_{k=-\infty}^{\infty} x[k] \sum_{n=-\infty}^{\infty} \delta[n - kL] e^{-j\omega n} = \sum_{k=-\infty}^{\infty} x[k] e^{-j\omega kL} = X(\omega L)$$

[code 구현]

`X_e = fft(x_e)`로 Zero-padding된 신호 `x_e`의 FFT를 계산한다.

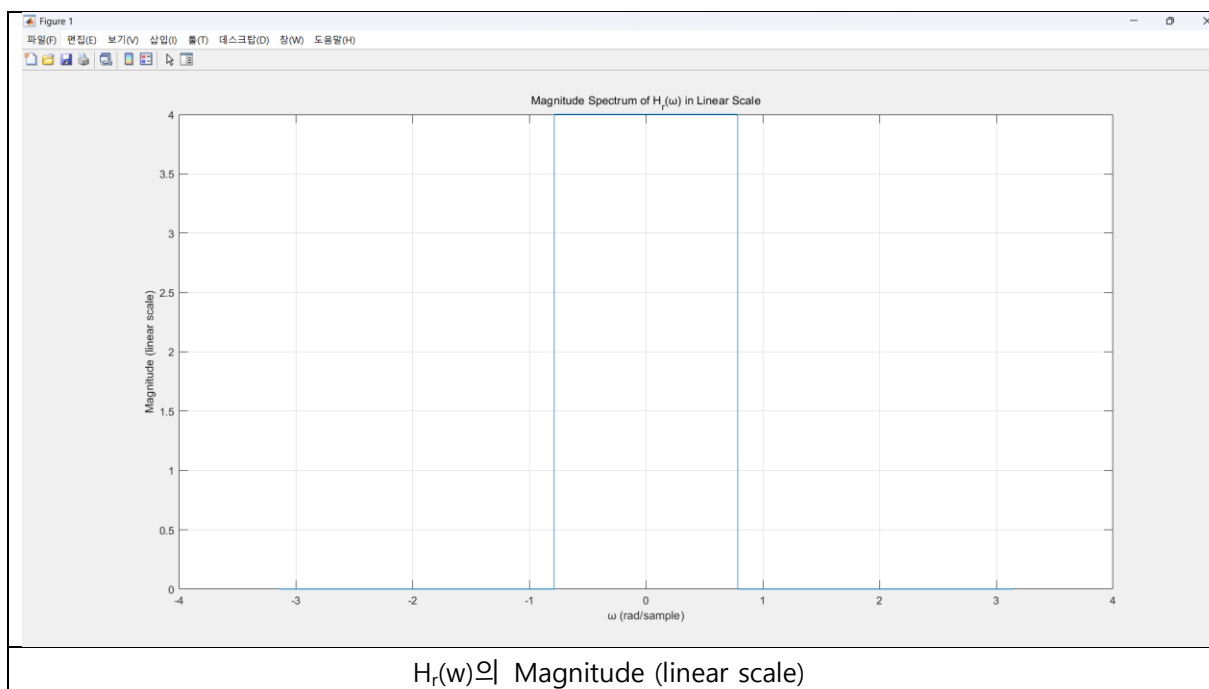
`X_e_shifted = fftshift(X_e)`를 사용하여 시각적으로 이해하기 쉽게 FFT 결과의 주파수 성분을 중앙으로 정렬한다.

`abs(X_e_shifted)`를 통해 `X_e`의 Magnitude를 계산하고, 이를 작은 크기를 가지는 데이터의 영향 또한 관찰하기 위해 log scale로 변환한다.

`upsample_omega = linspace(-pi, pi, L*n)`로 ω 축을 생성한다. `L*n`은 Zero-padding된 신호의 길이를 반영하여 생성된 ω 축이다.

2. Up-sampling을 하기 위해 적절한 Reconstruction Filter $h_r[n]$ 의 Frequency Response $H_r(\omega)$ 를 gain과 cutoff frequency에 주의하여 설계합니다. 그리고 설계한 $H_r(\omega)$ 에 Inverse Fourier Transform을 적용하여 시간축에서 필터를 구현합니다. 이때 Time domain에서 Ideal low-pass filter는 $(-\infty, \infty)$ 의 범위를 갖기에, 시간축에서 완전하게 구현할 수 없습니다. 따라서 Ideal low-pass filter $h_r[n]$ 를 각각 아래의 식과 같이 **5 points, 13 points rectangular** 함수 $w_5[n], w_{13}[n]$ 로 각각 **windowing**하여 $h_{r5}[n], h_{r13}[n]$ 를 구현합니다.

2-1. $H_r(\omega)$ 를 알맞게 설계한 뒤, $H_r(\omega)$ 의 Magnitude 그래프를 **log scale이 아닌 일반적인 linear scale**로 그리고, 설명하세요.



[$H_r(\omega)$ Filter 설명]

앞서 언급했듯이 $L=4$ 이기에 $X_e(\omega)$ 는 $-\pi/4 \sim \pi/4$ 까지가 의미가 있는 정보이고, 그 외는 중복되는 정보이다. 따라서, Low Pass Filter를 이용하여 의미 있는 정보만 남긴다. 하지만, 1-5에서 언급했듯이, $X_e(\omega)$ 와 $X(\omega)$ 의 magnitude는 같지만, $X_e(\omega)$ 는 $-\pi/4 \sim \pi/4$ 까지만 사용할 것이므로, power를 유지시켜줘야 한다. 따라서, Power를 4배로 할 수 있는 LPF를 설계해야 한다.

1) Filter의 크기: 4 ($L=4$ 이기에,)

2) 주파수 대역: $-\pi/4 \sim \pi/4$

[code 구현]

`upsample_n = L * length(x)`로 Zero-padding된 신호의 샘플 수와 똑같이 Filter의 샘플 수를 설정한다.

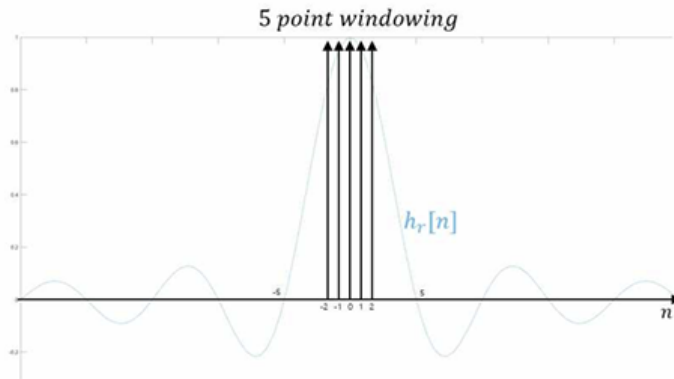
`upsample_omega = linspace(-pi, pi, upsample_n)`를 사용하여 w 축을 생성한다. 주파수 범위는 $-\pi$ 에서 π 까지이며, Zero-padding된 신호와 샘플링 수를 동일하게 맞춰 `upsample_n`개의 샘플을 사용하였다.

`H_r = zeros(upsample_n, 1)`로 $H_r(w)$ 라는 필터를 0으로 채운 배열로 초기화한다.

`middle_index = floor(upsample_n / 2) + 1`로 Filter의 중심 index를 계산한다. 필터의 중심은 Zero-padding된 신호의 주파수 성분에서 0 Hz에 해당하는 부분으로, 현재 샘플 수가 짝수로 설정되어 있으므로, 중심 index = $N/2 + 1$ 로 설정된다. 이 중심 index를 정확하게 계산하여야, ifft를 하였을 때, symmetry property의 Duality에 의해 $h_r[n]$ 이 Real 신호로 변환된다.

`start_index`와 `end_index`를 통해 Filter의 주파수 대역 범위를 계산한다. (샘플 수/ 2)가 실제 w 축에서 π 만큼의 범위를 의미한다. 따라서, 중심 index를 기준으로, $-\pi/4 \sim \pi/4$ 에 해당하는 샘플들에 L (= 크기 4)를 할당한다.

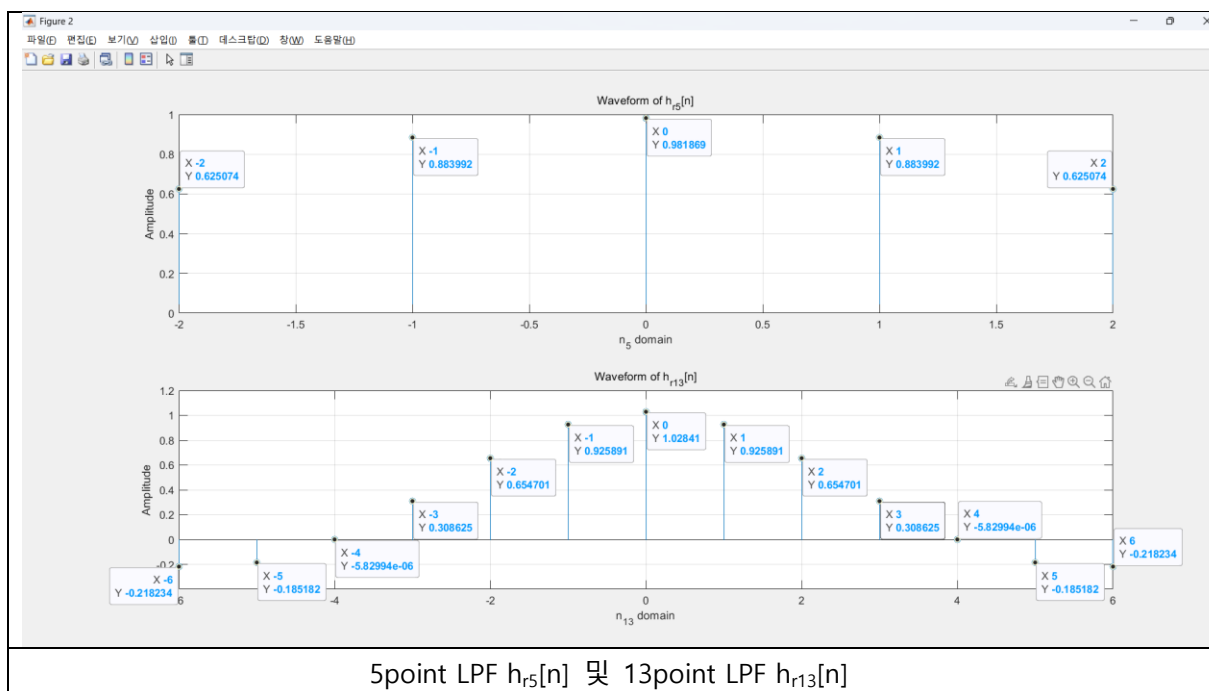
2-2. 주파수 축에서 구현한 $H_r(\omega)$ 를 시간축에서의 $h_r[n]$ 로 변환한 뒤 아래와 같이 $w_5[n], w_{13}[n]$ 로 각각 windowing한 $h_{r5}[n], h_{r13}[n]$ 을 구하세요. 그리고 둘의 파형을 그리세요. 단, $h_{r5}[n], h_{r13}[n]$ 의 모든 coefficient 합은 L 이 되도록 normalize 해야 합니다.



$$h_r[n] = IDFT(H_r(\omega))$$

$$w_5[n] = \begin{cases} 1, & n = -2, -1, 0, 1, 2 \\ 0, & \text{otherwise} \end{cases}$$

$$h_{r5}[n] = w_5[n]h_r[n], \quad h_{r13}[n] = w_{13}[n]h_r[n]$$



5point LPF $h_{r5}[n]$ 및 13point LPF $h_{r13}[n]$

[LPF $h_{r5}[n]$ 및 LPF $h_{r13}[n]$ 설명]

LPF의 pass band가 $-\pi/4 \sim \pi/4$ 까지 이므로, IFFT를 사용하여 변환한 $h_r[n]$ 은 $n = 4*m$ 마다 zero crossing이 일어난다. 즉, 5point windowing신호는 zero crossing이 되기 전의 5샘플, 13point windowing은 zero crossing을 2개 포함하는 13샘플로 나타난다. 더불어, windowing을 하여도, 신호의 power는 유지되어야 하기에 샘플의 크기의 합은 L (크기 = 4)로 정규화 시켜준다.

[code 구현]

$H_{r_original} = \text{ifftshift}(H_r)$ 를 하여 Filter $H(w)$ 의 0 rad/sec를 샘플 0에서 시작하도록 중앙 정렬한다.

그 후, $h_r = \text{ifftshift}(\text{ifft}(H_{r_original}))$ 는 H_r 을 IFFT를 통해 주파수 영역에서 n domain으로 변환한 후, 다시 중심을 이동시켜 샘플의 중앙에 sinc함수의 peak가 위치하도록 한다.

% $H_r(w)$ 의 0 rad/sec를 샘플 0에서 시작하도록 변환

```
1.  $H_{r\_original} = \text{ifftshift}(H_r);$ 
```

% $H_r(w)$ 를 n domain으로 변환 후 다시 중심을 이동

```
2.  $h_r = \text{ifftshift}(\text{ifft}(H_{r\_original}));$ 
```

위 2개의 코드가 2-2에서 주의해야하는 point이다. 만약 $H_{r_original}$ 을 구하지 않고 ifft를 하게 되면, ifft()함수 입장에서 이 신호는 0 rad/sec를 중심으로 passband가 형성 되어있는 LPF가 아니라, $-\pi$ 와 π 부근에 passband가 형성되어 있는 High Pass filter와 같이 인식하여, 잘못 inverse fft를 계산할 것이다.

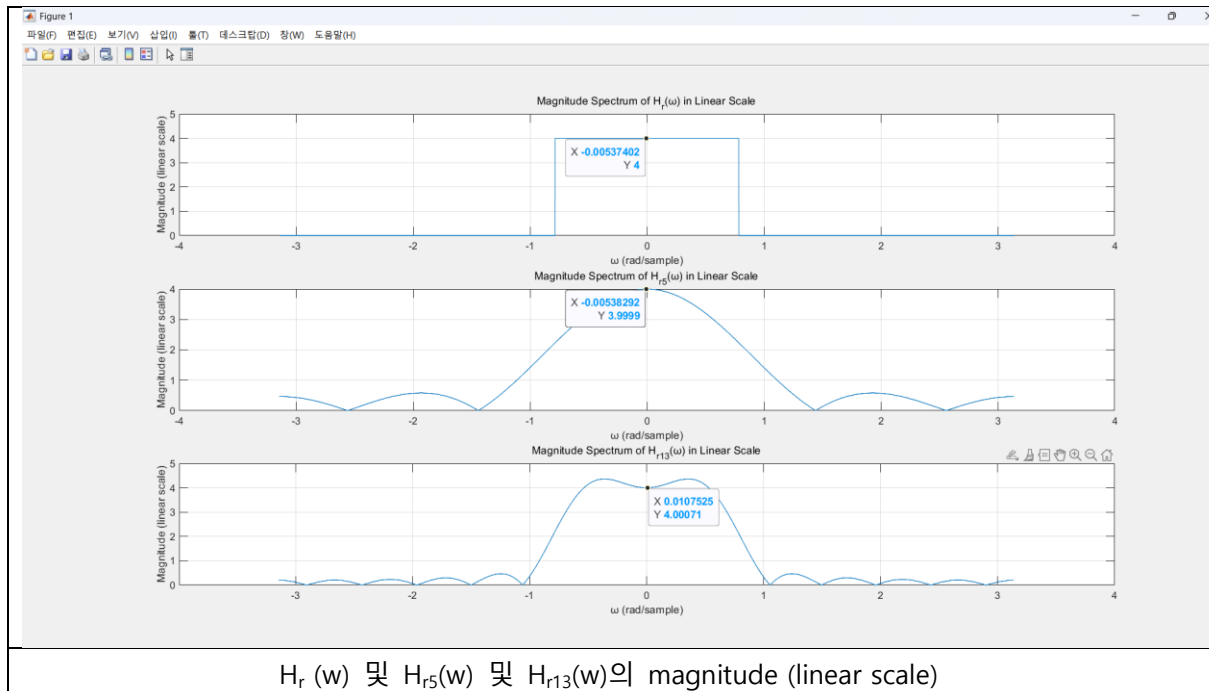
$n = -2, -1, 0, 1, 2$ 에 각각 크기가 1인 impulse를 할당하여, 5-point window filter(w_5)를 생성한다. 13-point window filter도 똑같이 진행한다.

$h_{r_5} = h_r(\text{middle_index} - 2 : \text{middle_index} + 2) .* w_5$ 를 하여, LPF의 중간 부분을 기준으로 5point만 추출하고, 각 값에 w_5 를 곱한다. 즉, 5point만 살아남고 나머지는 0이 된다.

$h_{r_13} = h_r(\text{middle_index} - 6 : \text{middle_index} + 6) .* w_{13}$ 도 마찬가지로 진행하여 결론적으로 13point만 살아남고 나머지는 0이 된다.

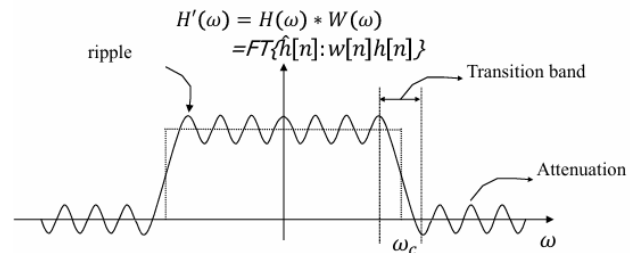
$h_{r_5} = h_{r_5} * L / \text{sum}(h_{r_5})$ 및 $h_{r_13} = h_{r_13} * L / \text{sum}(h_{r_13})$ 와 같이 모든 coefficients의 합이 L이 되도록 하여, power가 감소하지 않도록 한다. 원래 windowing전의 LPF h_r 은 coefficients의 합이 L이었다.

2-3. $h_{r5}[n]$, $h_{r13}[n]$ 의 Frequency Response $H_{r5}(\omega)$, $H_{r13}(\omega)$ 의 Magnitude를 log scale이 아닌 일반적인 linear scale로 그리고, $H_r(\omega)$ 와 비교하세요.



[H_r(ω) 및 H_{r5}(ω) 및 H_{r13}(ω)설명]

원래 Filter는 Ideal한 Low Pass Filter이다. Ideal한 Filter는 n-domain에서 무한대의 범위를 가지기 때문에, 구현이 불가능하다. 따라서, 이를 windowing하여 구현가능한 신호로 만든다.



Ideal한 LPF를 non-ideal LPF로 만들었을 때, 다음을 신경 써야 한다.

- 1) Transition band (좁을수록 좋음)
- 2) Ripple (작을수록 좋음)
- 3) Attenuation (0에 가까울수록 좋음)

먼저 5point windowing한 Filter를 살펴본다. 5point라는 것은 5개의 sample만 메모리에 저장한다는 것으로 실제 Ideal LPF에 비하면 매우 적은 메모리를 사용한다. 대신, trade off로 Filter의 성능이 급격히 안 좋아진다. 현재 w-domain에서 파형을 보면, transition band가 매우 넓은 것을 확인할 수 있다. 더불어, Attenuation 또한, 상대적으로 큰 값을 가진다. Ripple은 없어 보이지만, 이는 ripple이 너무 크기 때문에 ripple의 주기가 보이지 않고, 따라서, 없는 것과 같이 보이는 것이다. 즉, Filter의 성능은 매우 좋지 않다고 평가할 수 있다.

13point windowing한 Filter의 경우 5point windowing Filter에 비하여, ripple, transition band, attenuation이 모든 면에서 좋다. 하지만, 그럼에도 불구하고, Ideal Filter와 비교하였을 때, cut off 되는 구간의 모호하고, ripple과 attenuation이 여전히 크기에 잡음을 유발할 수 있다는 단점이 보

인다.

더 좋은 windowing 방법으로는 point수를 늘리거나, triangular windowing 또는, raised cosine filter를 사용하는 방법이 있다. 이는 sinc함수의 zero crossing 및 correlation을 고려하여 windowing하기에 더 좋은 Filter 성능을 보여준다.

[code 구현]

원래 Ideal Filter는 H_r_2 로 다시 만들어 plot, 5point LPF는 H_r_5 , 13point LPF는 H_r_{13} 으로 표기했다.

$H_r_2 = \text{fftshift}(\text{fft}(h_r, \text{length}(H_r)))$ 는 전체 필터 $h_r[n]$ 을 주파수 영역으로 변환한 결과이다. $\text{fft}()$ 를 통해 주파수 영역으로 변환하고, $\text{fftshift}()$ 로 중심 정렬하였다.

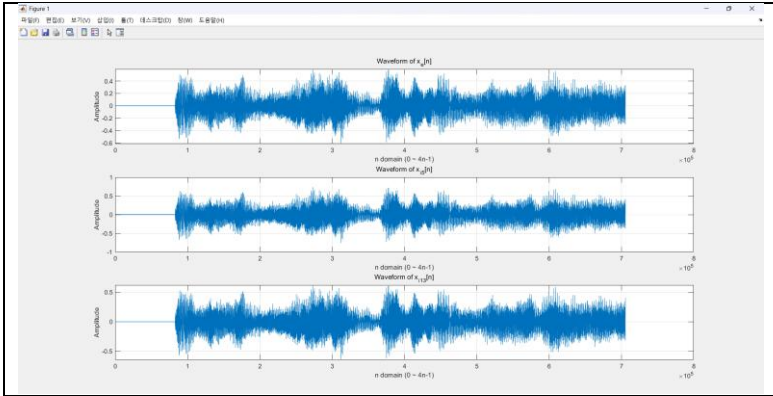
$H_r_5 = \text{fftshift}(\text{fft}(h_r_5, \text{length}(H_r)))$ 는 5-point window가 적용된 $h_r_5[n]$ 의 주파수 응답을 계산한 것이다. 정확한 비교를 위해 원래의 필터 길이 $\text{length}(H_r)$ 에 맞춰 zero-padding하여 비교가 쉽도록 파형을 plot하였다.

$H_r_{13} = \text{fftshift}(\text{fft}(h_r_{13}, \text{length}(H_r)))$ 도 마찬가지로, 13-point window가 적용된 $h_r_{13}[n]$ 의 주파수 응답을 계산한다. 이 또한, $\text{length}(H_r)$ 에 맞춰 zero-padding하여 비교가 쉽도록 파형을 plot하였다.

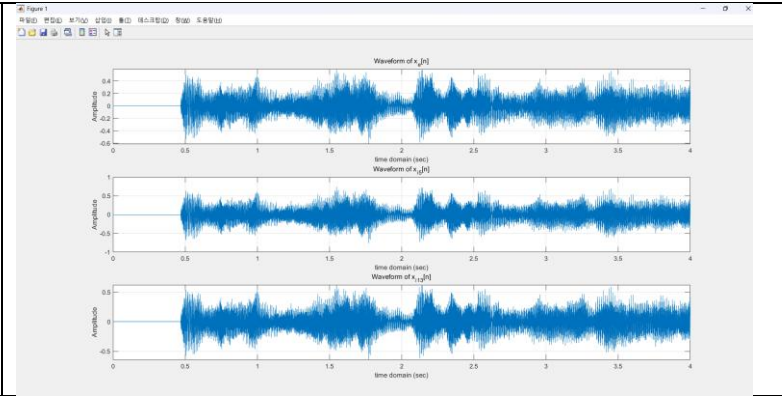
3. 1번 문제에서 Zero-padding 된 신호 $x_e[n]$, 2번 문제에서 얻은 Reconstruction filter $h_r[n]$ 를 통해, Up-sampling 된 신호 $x_i[n]$ 을 다음과 같이 얻을 수 있습니다.

$$x_i[n] = x_e[n] * h_r[n]$$

3-1. 신호 $x_e[n]$ 을 Reconstruction Filter $h_{r5}[n]$, $h_{r13}[n]$ 을 이용하여 **time domain**에서 filtering을 진행한 신호 $x_{i5}[n]$, $x_{i13}[n]$ 을 구하세요. 그리고 둘의 파형을 그리세요.
(※ 아래 Hint ④를 참고하세요.)



n domain에서 $x_e[n]$, $x_{i5}[n]$, $x_{i13}[n]$ 파형



t domain에서 $x_e[n]$, $x_{i5}[n]$, $x_{i13}[n]$ 파형

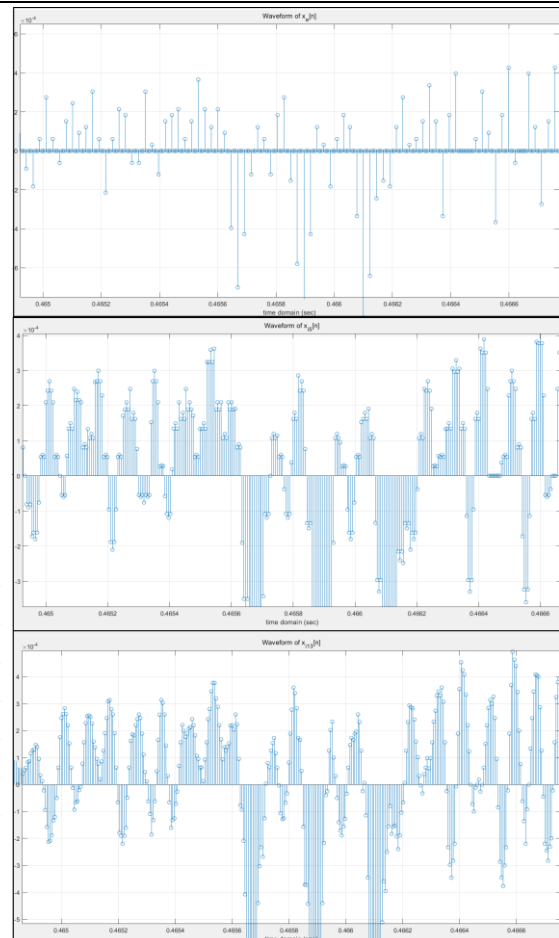
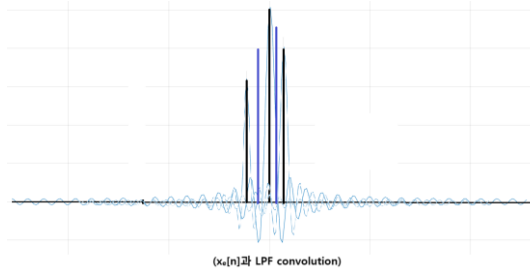
$[x_e[n]$, 파형 설명]

$x_e[n]$ 은 zero padding된 신호이고, $x_{i5}[n]$, $x_{i13}[n]$ 는 각각, 5point, 13point로 자른 LPF와 n-domain에서 convolution을 수행하여 나온 output신호이다.

오른쪽 그림을 보면 $x_e[n]$ 은 zero padding이 그대로 있어, 신호가 매끄럽지 못하다. 하지만, LPF와 convolution된 $x_{i5}[n]$, $x_{i13}[n]$ 은 신호와 신호 사이에 reconstruct가 일어나, 그 공간을 채워주고 있는 것을 볼 수 있다. 아래 그림은 $x_e[n]$ 과 LPF convolution이 일어나고 있는 모습을 표현한다. 아래와 같은 원리로 Interpolation이 완료된다. (파란 line: reconstruct에 의해 생긴 신호)

더불어, $x_{i5}[n]$, $x_{i13}[n]$ 을 비교해 보면, $x_{i13}[n]$ 이 상대적으로 더 매끄럽게 reconstruct가 된 것을 알 수 있다. 이를 통해 Ideal LPF에

가까울수록, reconstruct 성능이 좋은 것을 확인할 수 있다.



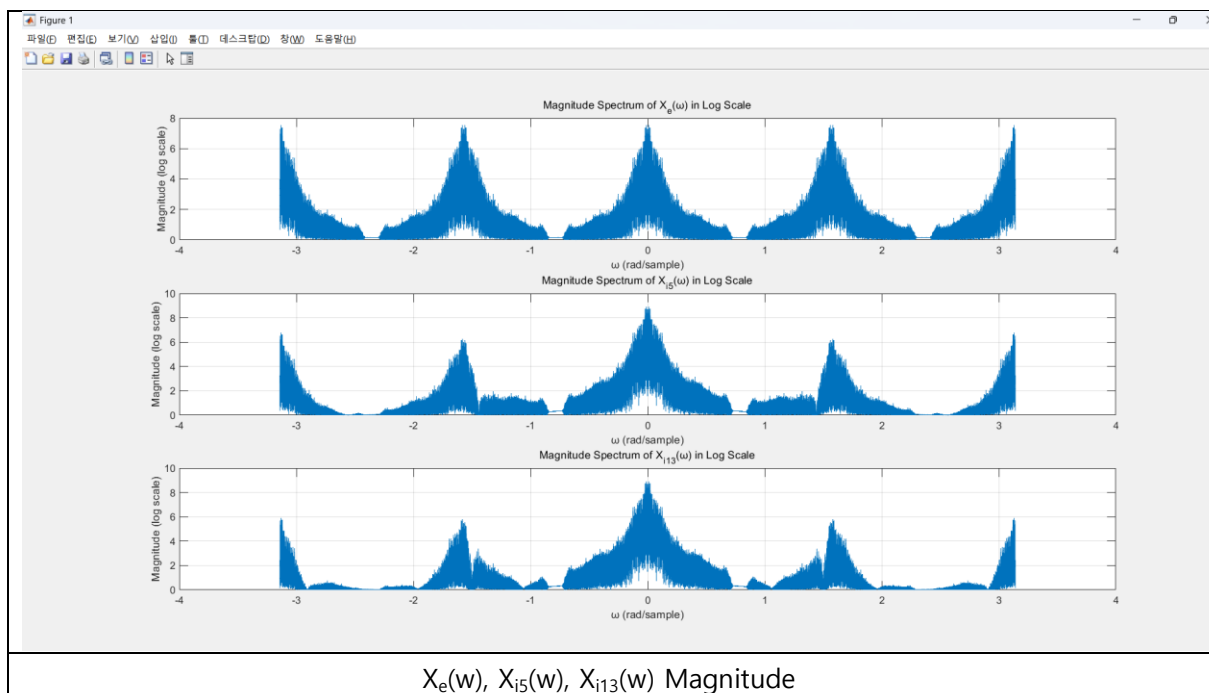
[code 구현]

$x_{i5} = \text{conv}(x_e, h_{r5}, \text{'same'})$ 은 신호 $x_e[n]$ 과 LPF $h_{r5}[n]$ 을 convolution한다. same 옵션은 결과 신호가 원본 신호와 같은 길이가 되도록 자른다.

이를 통해, FFT시, $X_e(\omega)$ 와 같은 샘플의 크기를 가질 수 있다.

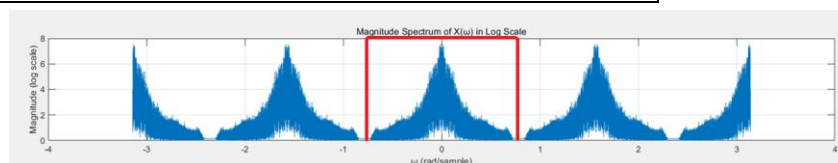
$x_{i13} = \text{conv}(x_e, h_{r13}, \text{'same'})$ 도 같은 방식으로 $x_e[n]$ 과 $h_{r13}[n]$ 을 convolution하여 $x_{i13}[n]$ 을 구한다.

3-2. 문제 3-1에서 filtering한 신호 $x_{i5}[n]$, $x_{i13}[n]$ 의 Spectrum $X_{i5}(\omega), X_{i13}(\omega)$ 의 magnitude 그래프를 각각 log scale로 그리세요. 또 $X_e(\omega)$ 와 어떻게 다른지 각각 비교하세요.



[$X_e(\omega), X_{i5}(\omega), X_{i13}(\omega)$ 비교]

Ideal Low Pass Filter를 적용하면, 오른쪽 그림과 같이 빨간색 부분만, 잘려야 한다.



먼저, gain이 L (크기=4)였으므로, 파형의 peak부분 Magnitude가 $X_e(\omega)$ 에 비해 증가한 것을 확인할 수 있다. 이를 통하여, 원래 신호의 power가 보존되었음을 알 수 있다.

하지만, 앞서 살펴보았듯이, 이번에 사용한 LPF는 5point 및 13point로 windowing하여 만든 LPF이다. 따라서, Transition band, Ripple, Attenuation 모두 성능이 좋지는 않았고, 특히나, 5point LPF의 경우 성능이 더욱 안 좋게 파형이 그려졌다.

$X_{i5}(w)$, $X_{i13}(w)$ 를 보게 되면, logscale에서 가장 뚜렷하게 드러나는 것은 안 좋은 Attenuation에 의한 파형들이다. Attenuation에 의해 $-\pi/4 \sim \pi/4$ 이외의 구간에서도 값이 튀는 것을 확인할 수 있다. 특히, $X_{i5}(w)$ 에서는 이 값들의 크기가 상대적으로 더 컸다.

더불어, 제대로 안 떨어지는 넓은 Transition Band에 의해 신호가 0으로 떨어져야 하는 부분에 신호가 없어지지 않고, 작게 남아있는 모습이 보인다.

Ripple에 의한 영향은 현재 Magnitude를 그린 파형에서는 확인하기 어렵다. 하지만, 2-3번에서 각 Filter의 파형을 보며, ripple의 영향이 있을 것을 알았기 때문에, 소리를 세심하게 들어보면 원신호에 비해 볼륨이 일정하지 않을 것이다.

[code 구현]

$x_e = \text{fftshift}(\text{fft}(x_e, \text{upsample_n}))$ 는 Zero-padding된 신호 $x_e[n]$ 의 주파수 응답을 계산한다.

이 결과는 Interpolation 전의 신호가 주파수 영역에서 어떤 성분을 가지고 있었는지 보여준다.

$x_i_5 = \text{fftshift}(\text{fft}(x_i_5, \text{upsample_n}))$ 는 5-point Reconstruction Filter를 적용한 신호 $x_{i5}[n]$ 의 주파수 응답이다.

이 결과는 5point LPF로 interpolation했을 때, 얼마나 주파수 성분이 제대로 잘리고, 어느 부분은 왜곡되는지를 확인할 수 있다.

$x_i_13 = \text{fftshift}(\text{fft}(x_i_13, \text{upsample_n}))$ 는 13-point Reconstruction Filter를 적용한 신호 $x_{i13}[n]$ 의 주파수 응답이다. 이 또한, 13 point LPF로 interpolation했을 때, 얼마나 주파수 성분이 제대로 잘리고, 어느 부분은 왜곡되는지를 확인할 수 있다.

3-3. $x_{i5}[n]$, $x_{i13}[n]$ 을 청취하고 신호 $x_e[n]$ 과 어떤 차이가 있는지 서술하세요.

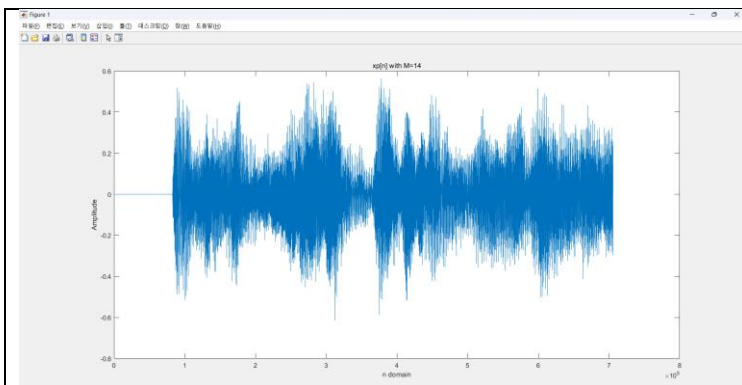
$x_e[n]$ 와 $x_{i5}[n]$, $x_{i13}[n]$ 을 비교했을 때, 가장 큰 차이는 소리의 음량이다. Interpolation을 진행할 때, 우리는 gain이 L (크기=4)인 LPF를 사용하였다. 이는 원신호의 power를 유지하기 위해서 이므로 $x_e[n]$ 에서 작았던 소리가, $x_{i5}[n]$, $x_{i13}[n]$ 에서는 원신호의 볼륨으로 회복된 것을 느낄 수 있었다. 더불어, interpolation하며 reconstruct된 신호들 덕분에, $x_e[n]$ 에 비해 약간 더 연속적이고 매끄럽게 들렸다.

사실 귀가 예민하지 않아서, Ideal 하지 않기에, 안 좋은 Transition band, Ripple, Attenuation를 가지는 LPF에 의한 영향은 잘 못 느꼈다. 예를 들어, 소리가 튀는 느낌이나, 이상한 소리가 들리는 이질감이 많이 느껴지지는 않았다. 하지만, $x[n]$ 과 비교하면, 확실히 소리의 질감이 더 매끄럽지 못한 느낌이 들었다.

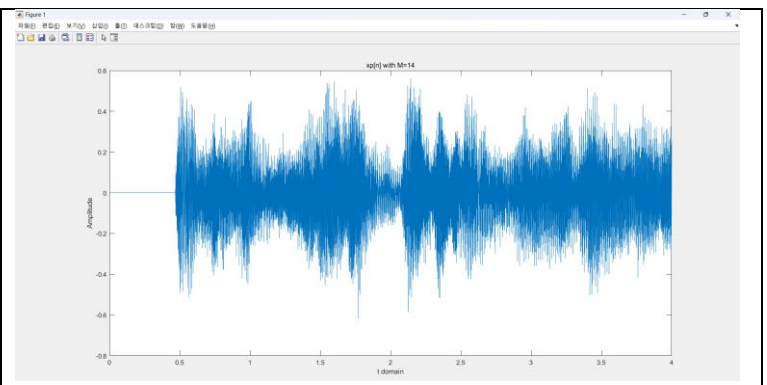
4. 다음은 Up-sampling된 신호를 Down-sampling하는 문제입니다. 이때, 문제 3번에서 얻은 $x_{i13}[n]$ 를 사용합니다.

4-1. 문제 3에서 얻은 신호 $x_{i13}[n]$ 에 대해, 아래의 식을 사용하여 $x_p[n]$ 을 구하고 파형을 그리세요. ($M=14$)

$$x_p[n] = \begin{cases} x_{i13}[n] & \text{for } n = Mk \\ 0 & \text{otherwise} \end{cases}$$



n domain에서 $x_p[n]$ 파형



t domain에서 $x_p[n]$ 파형

[$x_p[n]$ 파형 설명]

$x_p[n]$ 은 $x_{i13}[n]$ 에서 $n = kM$ 인 신호만 살리고, 나머지는 성분을 zero로 만들어버린 신호이다. 이 경우, 많은 신호들의 성분이 사라지고, M (크기=14)의 배수에 해당하는 성분들만 살아남는다.

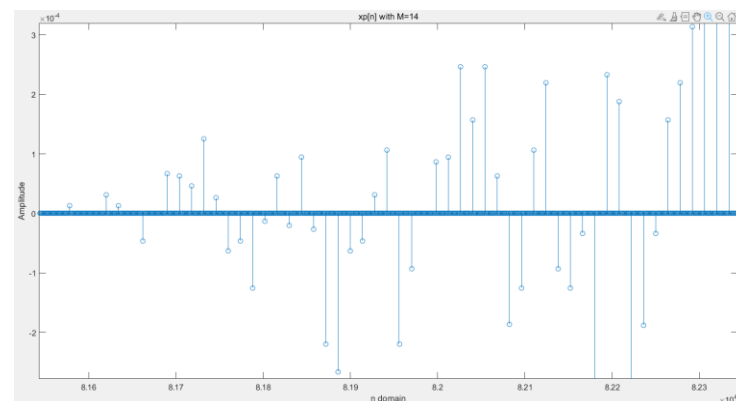
[code 구현]

`x_p = zeros(size(x_i_13))`를 통해 $x_{i13}[n]$ 과 동일한 샘플 수를 가지고 0으로 채워진 배열을 만든다.

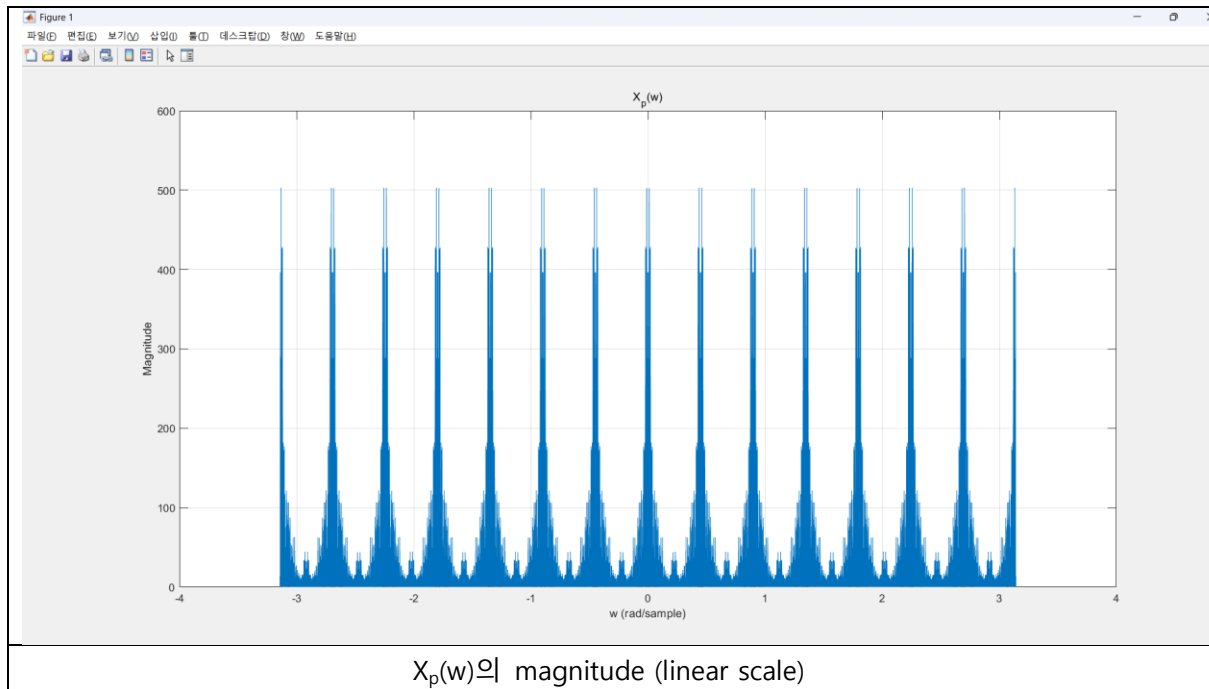
$M = 14$ 는 다운샘플링 비율로, 신호 성분을 14분의 1 수준으로 줄이겠다는 의미이다.

`x_p(1:M:end) = x_i_13(1:M:end)`는 $x_{i13}[n]$ 의 값을 M 간격으로 가져와서 $x_p[n]$ 에 할당하고, 나머지 값은 그대로 0으로 유지한다.

즉, $x_p[n]$ 은 14개의 샘플 중 하나만 남기고 나머지는 제거된 형태가 된다.



4-2. $x_p[n]$ 의 Spectrum $X_p(\omega)$ 의 magnitude 그래프를 그리고, 설명하세요.



[$X_p(\omega)$ 의 magnitude 설명]

$X_p(\omega)$ 는 $x_p[n]$ 을 FFT한 결과로, M (크기 14)만큼 샘플을 줄였을 때의 주파수 영역에서의 응답을 보여준다.

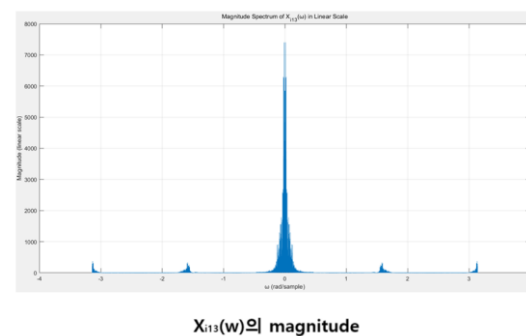
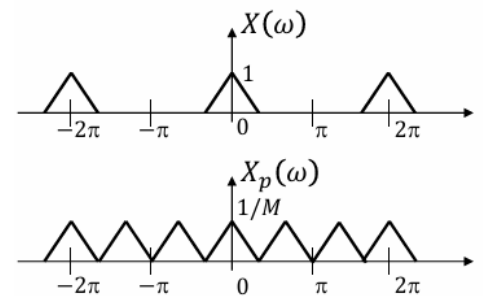
아래쪽 주파수 영역의 파형은 $X_{i13}(\omega)$ 의 magnitude이다.

Interpolation 및 Decimation을 하게 되면, 신호의 주파수 축이 변할 뿐만 아니라, 신호의 크기도 변하므로, 크기 및 주파수 대역을 짚고 넘어가야 한다.

1) 신호의 크기: $X_p(\omega)$ 의 신호의 크기는 $X_{i13}(\omega)$ 의 $1/M$ 배이다.

[약 7000 -> 500으로 감소]

2) 주파수 대역: $X_p(\omega)$ 의 경우 $-\pi \sim \pi$ 까지 동일한 Frequency Response가 14번 나온다. 즉, 14개 중 1개만 봤을 때, $-\pi/14 \sim \pi/14$ 까지의 대역을 차지한다고 볼 수 있다.



파시발 정리에 따르면 $-\pi \sim \pi$ 까지의 energy는 똑같다. 파형을 봤을 때도, $X_{i13}(\omega)$ 의 $-\pi \sim \pi$ 에서의 power와 $X_p(\omega)$ 의 $-\pi \sim \pi$ 에서의 power는 똑같이 나온다. 하지만, 의미 있는 정보가 존재하는 대역인 $-\pi/14 \sim \pi/14$ 를 보면 power가 $1/14$ 로 줄었다.

여기서, 주의 깊게 봐야 할 점은 현재 $X_p(\omega)$ 에 Aliasing이 발생했다는 것이다. 파형을 보면, $\pi/14$, $3\pi/14$, ... 에서 Aliasing이 발생하여, 신호가 튀는 것을 볼 수 있다.

[수식 유도]

$$p[n] = \sum_{k=-\infty}^{\infty} \delta[n - kM] = \sum_{k=0}^{M-1} a_k e^{j\frac{2\pi kn}{M}}$$

$$a_k = \frac{1}{M} \sum_{n=0}^{M-1} p[n] e^{-j\frac{2\pi kn}{M}} = \frac{1}{M} \sum_{n=0}^{M-1} \sum_{l=-\infty}^{\infty} \delta[n - kM] e^{-j\frac{2\pi kn}{M}} = \frac{1}{M} \sum_{n=0}^{M-1} \delta[n] e^{-j\frac{2\pi kn}{M}} = \frac{1}{M}$$

Thus,

$$p[n] = \frac{1}{M} \sum_{k=0}^{M-1} e^{j\frac{2\pi kn}{M}}$$

Now,

$$\begin{aligned} P(\omega) &= \mathcal{F}\{p[n]\} = \mathcal{F}\left\{\frac{1}{M} \sum_{k=0}^{M-1} e^{j\frac{2\pi kn}{M}}\right\} = \sum_{k=0}^{M-1} \frac{1}{M} \mathcal{F}\left\{e^{j\frac{2\pi kn}{M}}\right\} = \sum_{k=0}^{M-1} \frac{1}{M} 2\pi \sum_{l=-\infty}^{\infty} \delta(\omega - k\omega_s - 2\pi l) \\ &= \frac{2\pi}{M} \sum_{k=0}^{M-1} \delta(\omega - k\omega_s), \text{ (여기서, } \omega_s = \frac{2\pi}{M} \text{)} \end{aligned}$$

Now use convolution in w-domain and,

$$X_p(\omega) = \frac{1}{2\pi} X(\omega) * P(\omega) = \frac{1}{2\pi} X(\omega) * \sum_{k=0}^{M-1} \delta\left(\omega - \frac{2\pi k}{M}\right) = \frac{1}{M} \sum_{k=0}^{M-1} X\left(\omega - \frac{2\pi k}{M}\right)$$

[code 구현]

`xp = fft(x_p)` : n-domain 신호 `x_p`에 대한 Fourier Transform을 수행한다.

샘플의 개수는 $X_{i13}(\omega)$ 와 동일하다.

4-3. $x_p[n]$ 을 청취하고, 원래 신호 $x_{i13}[n]$ 와 어떤 차이가 있는지 서술하세요.

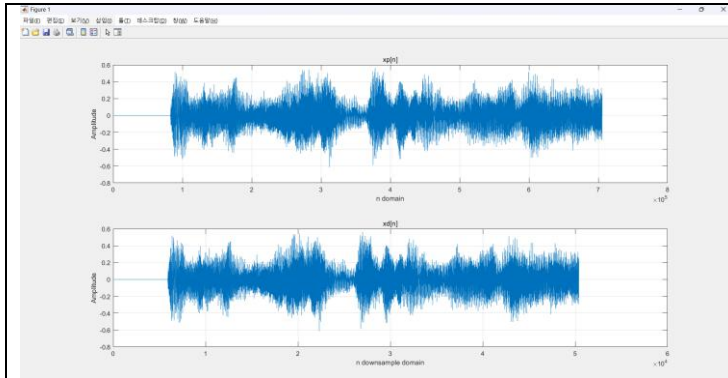
$x_p[n]$ 은 $x_{i13}[n]$ 에서 $n = kM$ 인 신호만 살리고, 나머지는 성분을 zero로 만들어버린 신호이다. 이를 주파수 영역으로 가져와서 살펴봤을 때, $\pi/14, 3\pi/14, \dots$ 에서 Aliasing이 발생하는 것을 볼 수 있었다.

이 Aliasing의 영향이 신호에서 바로 들렸다. 원래 음향 신호가 main으로 들리기는 하였지만, 뒤쪽에 지잉~거리는 이상한 소리가 지속적으로 계속 들렸다. 이 이상한 소리가 바로 Aliasing으로 인한, 원치 않은 소리가 발생한 결과이다.

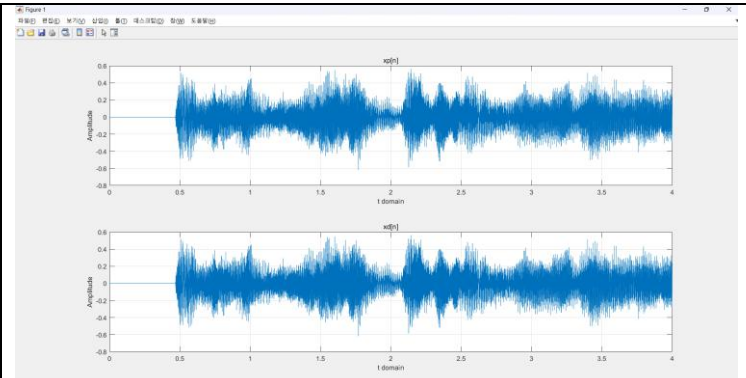
더불어, $x_e[n]$ 에서와 같은 이유로 소리의 음량이 차이가 났다. $x_{i13}[n]$ 은 원신호의 음량을 가진 신호로, 이를 14단위마다 살리고, 나머지는 0으로 만들었으니, 음량은 거의 14배 작게 들려야 한다. 주파수 영역으로 생각했을 때도 의미 있는 정보는 $-\pi/14 \sim \pi/14$ 에만 존재하고 그 파형의 크기는 $X_{i13}(\omega)$ 에 비해 1/14의 크기를 가졌다. 실제로도 음량이 매우 작게 나왔다.

4-4. Down-sampling을 하기 위해 문제 3에서 얻은 신호 $x_p[n]$ 에 대해, 아래의 식을 사용하여 $x_d[n]$ 의 파형을 그리세요. ($M=14$)

$$x_d[n] = x_p[Mn]$$



n domain에서 $x_p[n]$, $x_d[n]$ 파형



t domain에서 $x_p[n]$, $x_d[n]$ 파형

[$x_d[n]$ 설명]

$x_d[n]$ 은 샘플 수를 14배로 줄여 원 신호에서 0이었던 부분을 모두 없애고, 성분을 취한 부분만 남긴 신호이다. 즉, n-축을 compress시켰다고 생각할 수 있다.

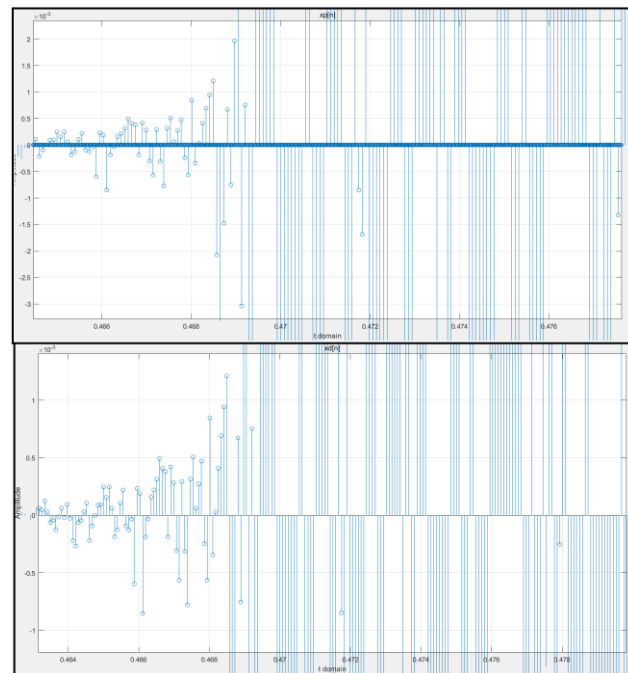
오른쪽 파형을 확인하면, 0인 값들이 전부 제거된 것을 볼 수 있다.

[code 구현]

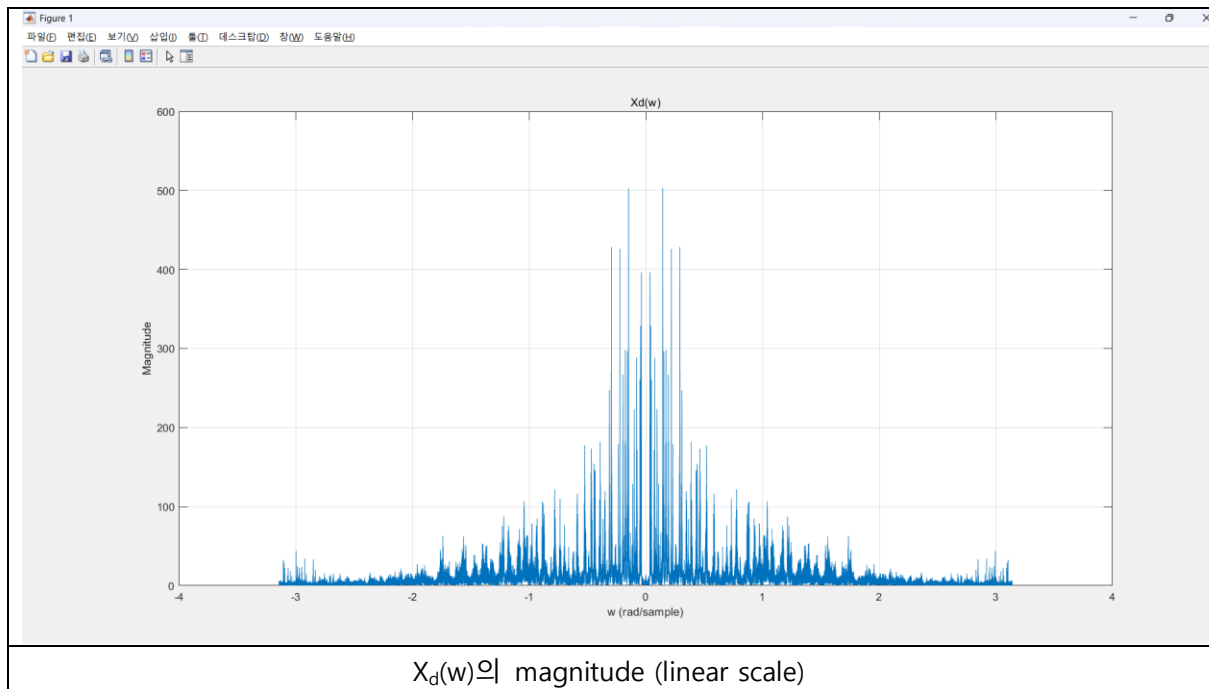
`x_d = zeros(ceil(size(x_p)/M)):x_p[n]의 1/M사이
즈의 0으로 초기화 된 x_d배열을 만든다.`

`x_d(1:1:end) = x_p(1:M:end):x_d[n]은 그 중에서 의
미 있는 값들만 추출한 다운샘플링 결과다.`

참고로, $x_p[n]$ 은 이전 단계에서 생성한, M 간격으로 값이 있는 신호이며 나머지는 모두 0이다.



4-5. $x_d[n]$ 의 Spectrum $X_d(\omega)$ 의 magnitude 그래프를 그리세요.



[$X_d(\omega)$ 설명]

n-domain에서 compress는 w-domain에서 축을 늘리는 것과 동일하다. 즉, $-\pi/14 \sim \pi/14$ 의 스펙트럼이 $-\pi \sim \pi$ 까지로 늘어난다.

1) 신호의 크기: $X_d(\omega)$ 의 신호의 크기는 $X_p(\omega)$ 와 동일하다.

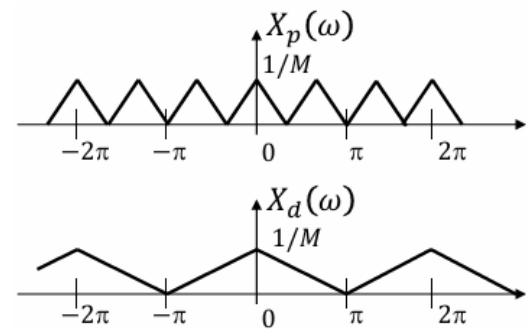
[500 -> 500으로 유지]

2) 주파수 대역: $X_p(\omega)$ 의 경우 $-\pi \sim \pi$ 까지 동일한 Frequency

Response가 14번 나온다. 즉, 14개 중 1개만 봤을 때, $-\pi/14 \sim \pi/14$ 까지의 대역을 차지한다고 볼 수 있다. 하지만, $X_d(\omega)$ 는 스펙트럼이 $-\pi \sim \pi$ 까지로 늘어났다.

Power의 관점에서 $X_p(\omega)$ 는 $-\pi/14 \sim \pi/14$ 까지만 의미있는 정보가 존재했고, 그 구간의 magnitude는 $X_{i13}(\omega)$ 의 $1/14$ 였다. $X_d(\omega)$ 의 magnitude 또한 $X_{i13}(\omega)$ 의 $1/14$ 이다. 하지만, 주파수 대역이 $-\pi \sim \pi$ 까지로 증가하였기에, power는 $X_{i13}(\omega)$ 와 동일해진다.

더불어, $X_p(\omega)$ 에서 나타났던 Aliasing이 고주파 대역으로 이동하여 남아있는 것을 확인할 수 있다.



[수식 유도]

$$X_d(\omega) = \sum_{n=-\infty}^{\infty} x_p[Mn]e^{-j\omega n} = \sum_{n=-\infty}^{\infty} x_p[n]e^{-j\omega n} = X_p\left(\frac{\omega}{M}\right)$$

[code 구현]

`Xd = fftshift(fft(x_d))`: x_d 를 fourier transform을 하고 알아보기 쉽게 파형의 중심을 그래프의 가운데로 옮긴다.

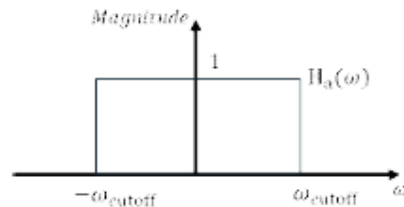
더불어, 샘플 수는 down sampling을 하였으니 $X_{i13}(w)$ 에 비해 $1/M$ 으로 줄어든다.

4-6. $x_d[n]$ 을 청취하고, 원래 신호 $x_p[n]$ 과 어떤 차이가 있는지 서술하세요.

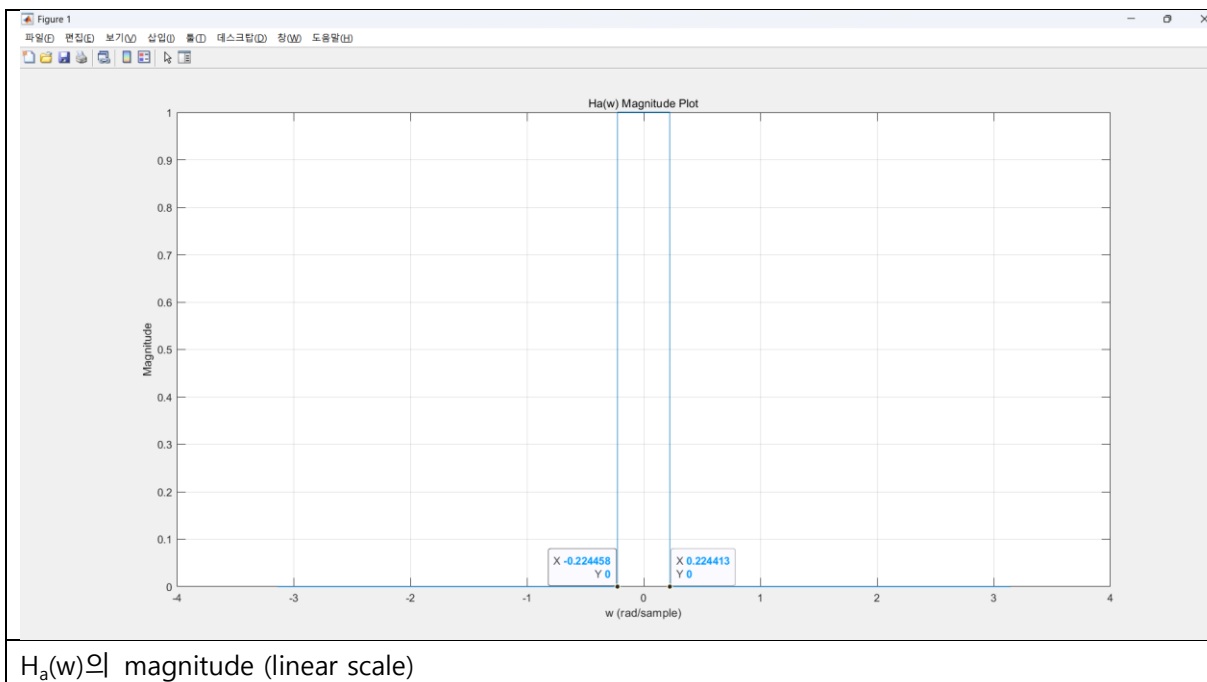
먼저, 소리의 음량이 커졌다. 이는 4-5에서 언급했듯이, 원신호의 power를 되찾았기 때문이다. 더불어, $x_p[n]$ 에서 나타났던 Aliasing이 $x_d[n]$ 에서는 더 짚는 소리로 변하였다. 이는 Aliasing이 발생한 주파수 대역이 고주파로 이동했기 때문이라고 생각한다.

원래 down sampling시 없었던 고주파 대역에 신호가 생기므로, 소리의 해상도가 증가하는 효과가 있어야 하는데, Aliasing으로 인해 해상도 증가하는 것을 느끼기 보다는 짚는 소리가 고주파 영역에서 계속 들리는 느낌이었다.

5. Down-sampling을 진행할 때 나타나는 Aliasing 문제를 해결하기 위해 **Anti-aliasing low pass filter** $H_a(\omega)$ 를 설계하고 적용하는 과정에 대한 문제입니다. 필터 $H_a(\omega)$ 를 직접 설계하고, 이 필터로 신호 $x_{i13}[n]$ 를 **frequency domain**에서 filtering 한 뒤, down-sampling을 수행하여 신호 $x_f[n]$ 를 얻으세요.



- 5-1. $H_a(\omega)$ 의 magnitude 그래프를 log scale이 아닌 일반적인 linear scale로 그리고, 설명하세요.



[$H_a(\omega)$ 설명]

4-2번에서 확인하였던 Aliasing의 영향을 없애기 위하여 Anti-Aliasing LPF를 설계한다. Aliasing을 없애는 용도로만 설계하는 것으로 gain = 1이다. Anti-Aliasing Filter에서 가장 중요한 것은 Down Sampling을 하기 전에 Aliasing이 일어날 대역을 미리 0으로 만드는 것이다.

Aliasing이 일어날 대역은 M에 의해 결정된다. Down sampling하기 전의 신호가 $-\pi/M \sim \pi/M$ 의 범위를 넘어서 주파수 성분을 가지고 있으면, 그 부분은 Aliasing을 일으킬 것이다.

따라서, LPF를 이용하여 $-\pi/M \sim \pi/M$ 의 범위의 주파수 성분만 남기고 나머지는 0으로 만든다.

[code 구현]

```
1. H_a = zeros(upsample_n, 1);
2. middle_index = floor(upsample_n / 2) + 1;
3. start_index = middle_index - floor(upsample_n / M / 2);
4. end_index = middle_index + floor(upsample_n / M / 2);
5. H_a(start_index:end_index) = 1;
```

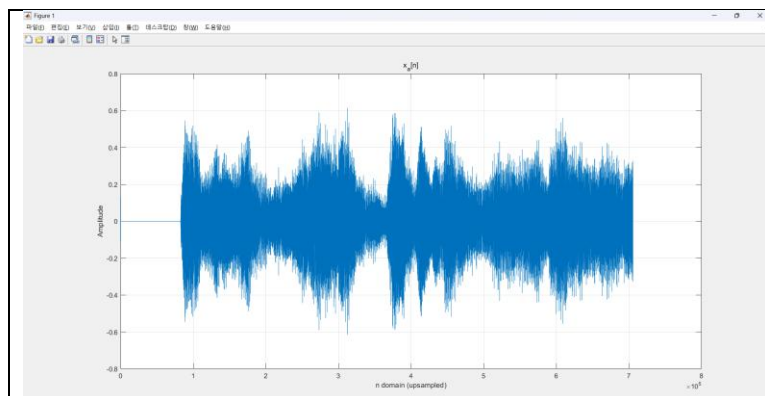
2-1번과 거의 동일하게 진행하였다. 차이점은 주파수 대역과, gain이다. 즉, 이번에는 $-\pi/M \sim \pi/M$ 범위에 1의 gain을 갖도록 설계하였다.

참고로, $X_{i13}(\omega)$ 를 LPF 취할 것이기에 $X_{i13}(\omega)$ 와 같은 샘플 수를 이용하였다

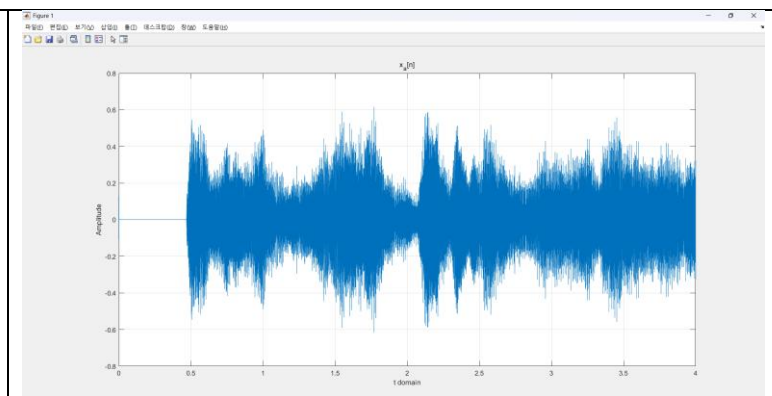
5-2. 문제 5-1에서 설계한 Anti-aliasing lowpass filter를 사용하여 아래의 식과 같이 신호 $x_{i13}[n]$ 를 filtering하여 신호 $x_a[n]$ 을 구하고, 파형을 그리세요.

$$X_a(\omega) = H_a(\omega)X_{i13}(\omega)$$

$$x_a[n] = IDFT(X_a(\omega))$$



n domain에서 $x_a[n]$ 파형



t domain에서 $x_a[n]$ 파형

[$x_a[n]$ 설명]

2번 문제와는 다르게 이번에는 Fourier Transform의 Convolution Property를 이용하여 $X_a(\omega)$ 를 구한다.

$x_a[n]$ 은 $x_{i13}[n]$ 에 LPF를 취한 것이기 때문에, $-\pi/14 \sim \pi/14$ 을 넘어서는 범위의 주파수 성분이 없어졌을 것이다. 따라서, n-domain 또는 t-domain에서는 육안으로 확인하기 어렵지만, $x_a[n]$ 은 매우 저주파 대역만 남아있는 신호가 된다.

n-domain 또는 t-domain에서 확인가능한 것은 음량의 세기이다. $x_a[n]$ 과 $x_{i13}[n]$ 모두 비슷한 Amplitude를 가지고 있는 것을 확인할 수 있고, 비슷한 소리가 날 것임을 알 수 있다.

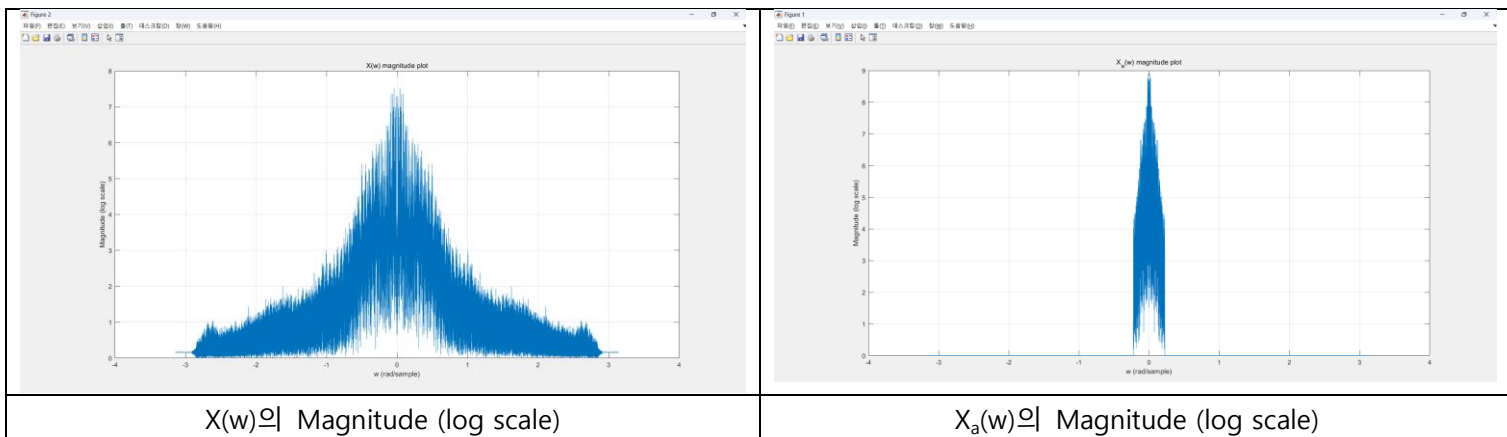
[code 구현]

$X_a = H_a .* X_i_13$: n-domain에서 convolution은 주파수 영역에서 곱셈이므로, 좌측 코드와 같이 LPF를 취한다.

$x_a = \text{ifft}(X_a) : 5-2$ 에서 분석하는 것은 n-domain $x_a[n]$ 이므로 Inverse Fourier Transform을 한다.

샘플 수는 X_i_13 과 동일해야 하므로, upsampling때 사용했던 샘플 수를 사용한다.

5-3. $x_a[n]$ 의 Spectrum $X_a(\omega)$ 의 magnitude를 log scale로 그리고 원본 Spectrum $X(\omega)$ 와 비교하세요.



[$X(\omega)$ 및 $X_a(\omega)$ 설명]

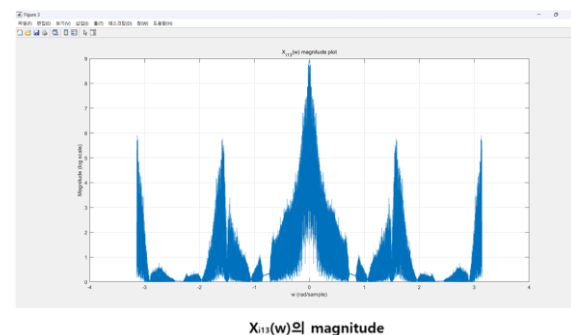
$X_a(\omega)$ 는 $X(\omega)$ 를 Interpolation한 결과 $X_{i13}(\omega)$ 를 $-\pi/14 \sim \pi/14$ 의 범위만 남기고 자른 것이다.

즉, $X_{i13}(\omega)$ 의 magnitude는 $X(\omega)$ 의 L (크기=4)배이었고, 의미 있는 정보(= $X(\omega)$ 의 $-\pi \sim \pi$ 까지의 정보)는 주파수 영역에서 압축되어 $-\pi/4 \sim \pi/4$ 에 존재하였지만, 이를 gain이 1인 $-\pi/14 \sim \pi/14$ 가 pass band인 LPF로 자른 결과인 것이다. 이 과정으로 인해, 원신호의 고주파 영역에 있었던 정보들이 많이 사라졌다.

만약 $X(\omega)$ 를 Interpolation하지 않고, 그냥 LPF를 취하여 $-\pi/14 \sim \pi/14$ 만 남겼으면 어떤 상황이 발생하였을지 생각해본다.

만약, Interpolation을 하지 않고, LPF를 하였다면, 원신호의 해상도를 높여주는 여러 주파수 성분들이 없어졌을 것이다. 즉, 원신호의 $-\pi/14 \sim \pi/14$ 를 제외한 범위를 다 날려버리고 이는 매우 큰 정보의 손실을 유발한다. 즉, 원 신호의 sampling rate가 1/14가 되어버리며, Aliasing이 크게 발생할 텐데, 이를 막기위해서 많은 정보의 손실을 감수해야 한다는 것이다.

$X_a(\omega)$ 의 경우에는 4배 Interpolation을 하고 14배 decimation을 했기에, 2/7로 sampling rate가 줄었고, 1/14로 sampling rate를 줄일 때보다 정보의 손실은 상대적으로 훨씬 적었다.



[code 구현]

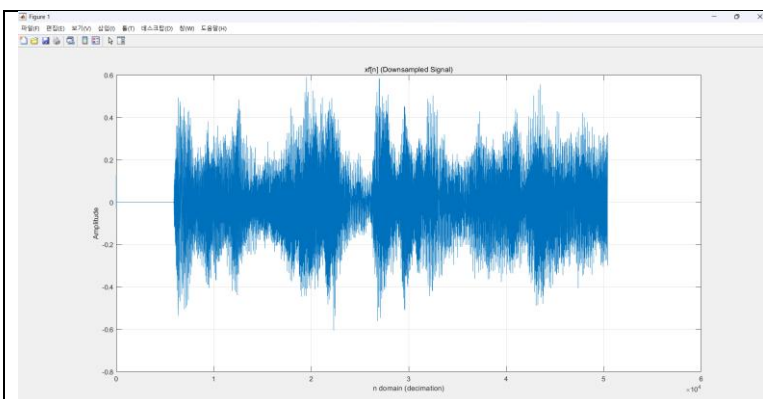
```
1. X_a = H_a .* X_i_13; % Xa(w) = multiply(Ha(w), X_i_13(w))
```

5-2에서 x_a 를 구현하기 위해 $X_a(w)$ 를 만들었다. 이 $X_a(w)$ 를 그대로 plot하였다.

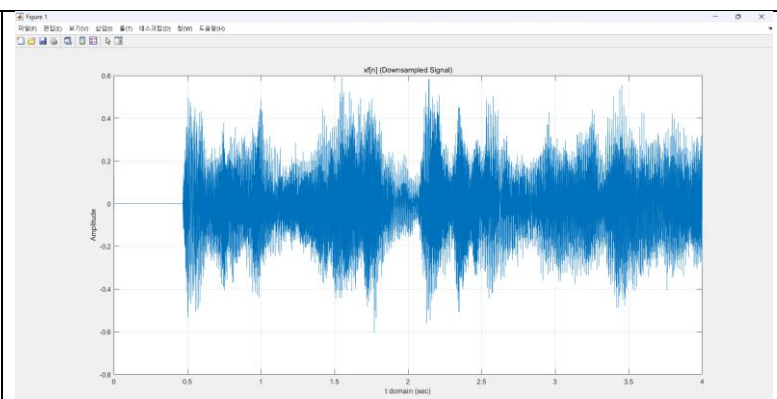
샘플 수는 (원 신호 샘플) * L / M로 설정하여야 한다. [Interpolation후 decimation이므로]

5-4. 신호 $x_a[n]$ 에 대하여 down-sampling을 수행한 $x_f[n]$ 을 얻고, 파형을 그리세요.
($M=14$)

$$x_f[n] = x_a[Mn]$$



n domain에서 $x_f[n]$ 파형

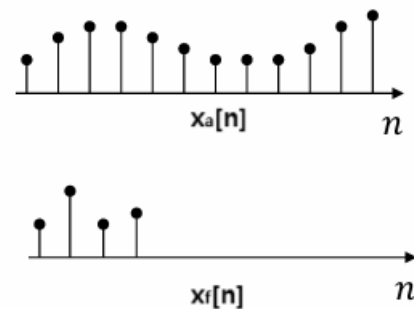


t domain에서 $x_f[n]$ 파형

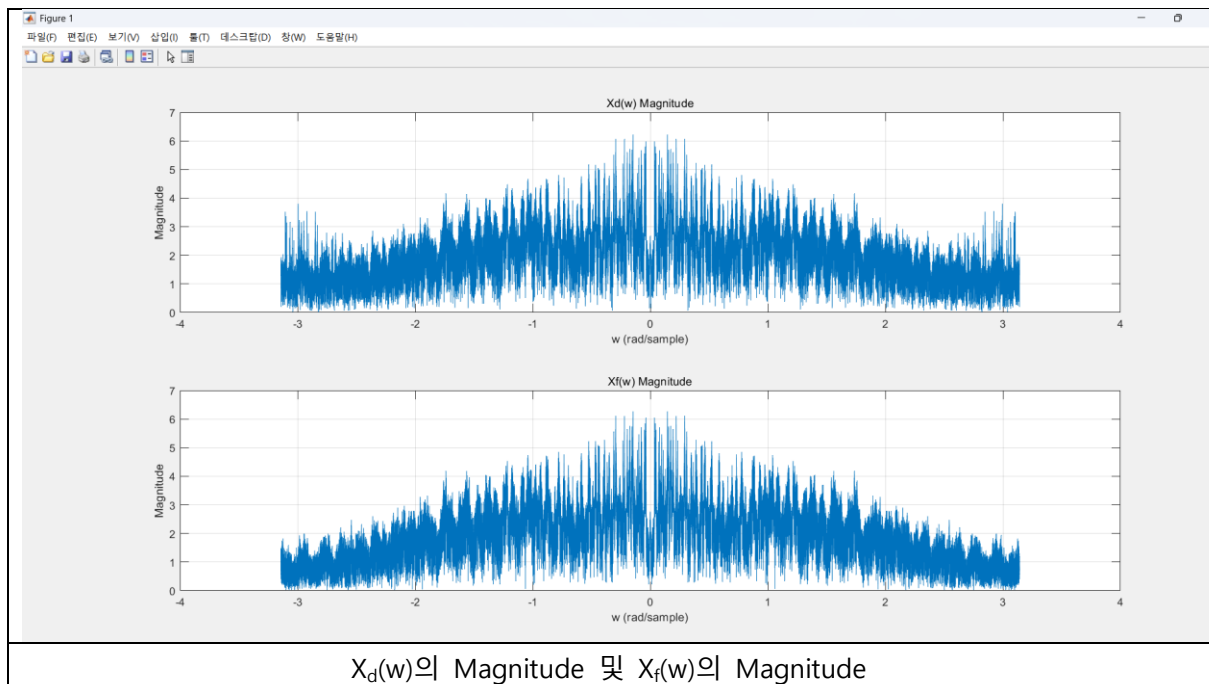
[$x_f[n]$ 설명]

$x_f[n]$ 은 $x_a[n]$ 샘플 수를 0부터 시작하여 14번째마다 샘플을 가져온 신호로 14배로 줄인 신호이다. 즉, n-축에서 정보를 일부분만 가져온 후 n-축을 compress시켰다고 생각할 수 있다. 오른쪽 그림은 이를 간단하게 그림으로 표기한 것이다.

[그림의 경우에는 14배 압축은 아니다.]



5-5. $x_f[n]$ 의 Spectrum $X_f(\omega)$ 의 magnitude를 log scale로 그리고 문제 4-5의 $X_d(\omega)$ 와 비교하세요.



$X_d(\omega)$ 의 Magnitude 및 $X_f(\omega)$ 의 Magnitude

[$X_d(\omega)$ 의 Magnitude 및 $X_f(\omega)$ 의 Magnitude 설명]

$X_d(\omega)$ 의 Magnitude를 보게 되면, $-\pi$ 와 π 부근에 파형이 튀는 것을 확인할 수 있다. 이것이 Aliasing에 의한 신호 변형인데, Anti-Aliasing Filter를 적용한 $X_f(\omega)$ 의 Magnitude는 $-\pi$ 와 π 부근에 신호에 Aliasing이 발생하지 않아 파형이 튀지 않는다.

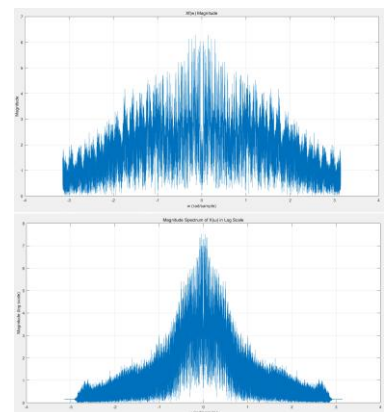
따라서, Anti-Aliasing Filter를 적용하여 만든 신호는 지잉~거리는 고주파 소리가 나지 않을 것으로 예상된다.

더불어, 두 파형 decimation과정을 통해 모두 주파수 축을 늘렸기에 $-\pi$ 와 π 까지 magnitude가 존재하는 것을 확인할 수 있다.

5-6. $x_f[n]$ 을 청취하고, Aliasing이 발생한 신호 $x_d[n]$ 과 어떤 차이가 있는지 서술하세요.

$x_d[n]$ 에서는 본래 찢어지는 소리가 배경에 존재했다. 이는 Aliasing이 발생한 주파수 대역 때문이었다. 하지만, Anti-Aliasing filter를 적용하여 Aliasing을 제거하였더니, 찢어지는 소리가 사라지고, 원신호의 깨끗한 음만 존재하였다.

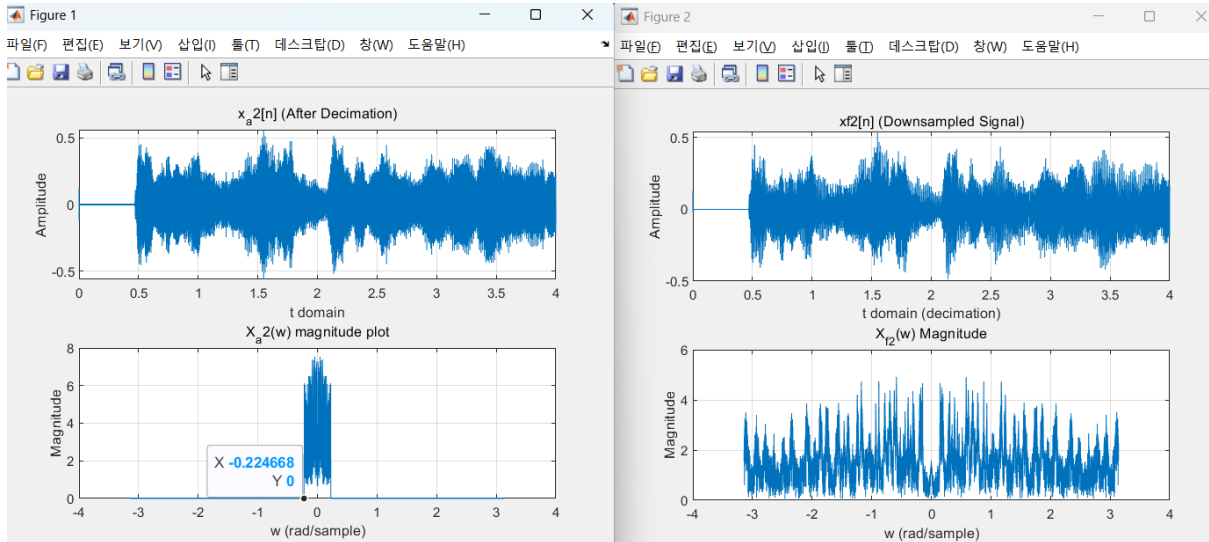
하지만, 원신호와 비교하면, 소리의 디테일이 부족한 느낌이 들었다. 이는 Decimation을 진행하면서 $-\pi/14 \sim \pi/14$ 의 범위만 LPF로 잘라버려 정보의 손실이 생겼기 때문이다. 따라서, 주파수 영역에서 magnitude 파형을 확인해보면, 원 신호의 파형과는 디테일이 다르게 나온다. 이는 최종적으로 L/M만큼 sampling rate가 변했을 때의 주파수 영역이 어떻게 달라지는지 시각적으로 보여준다.



6. Discussion

Interpolation을 진행하지 않고 바로 $M=14$ 로 decimation하였을 때

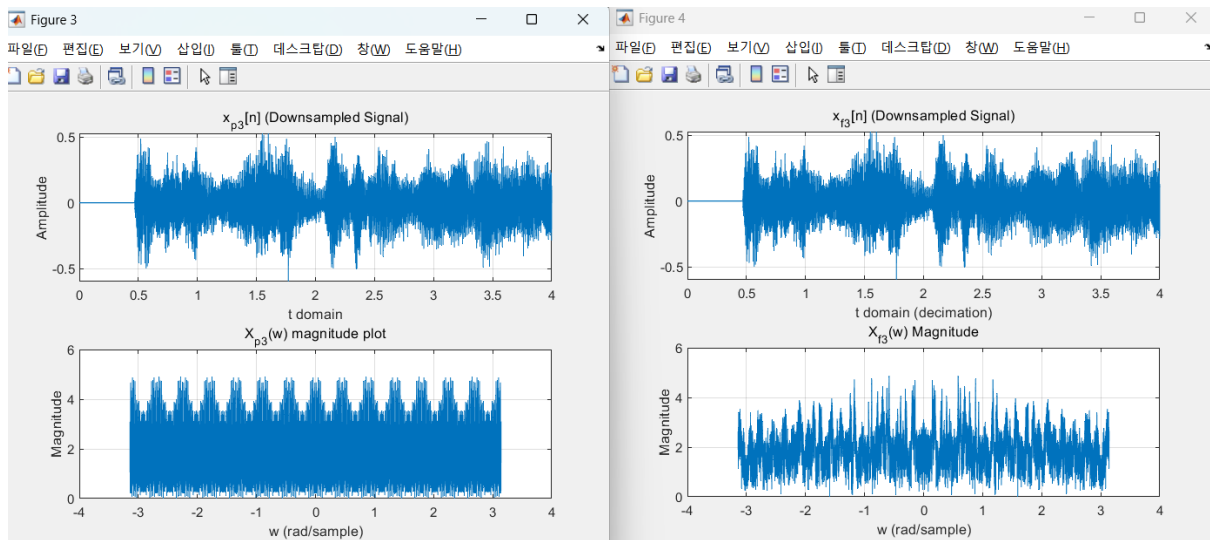
6-1. Anti-Aliasing Filter 사용하여 $M=14$ 로 decimation



Interpolation을 하지 않고 바로 Decimation을 하게 되면 Anti-Aliasing Filter에 의하여 기존, $X(w)$ 의 고주파 대역의 정보가 다 사라진다.

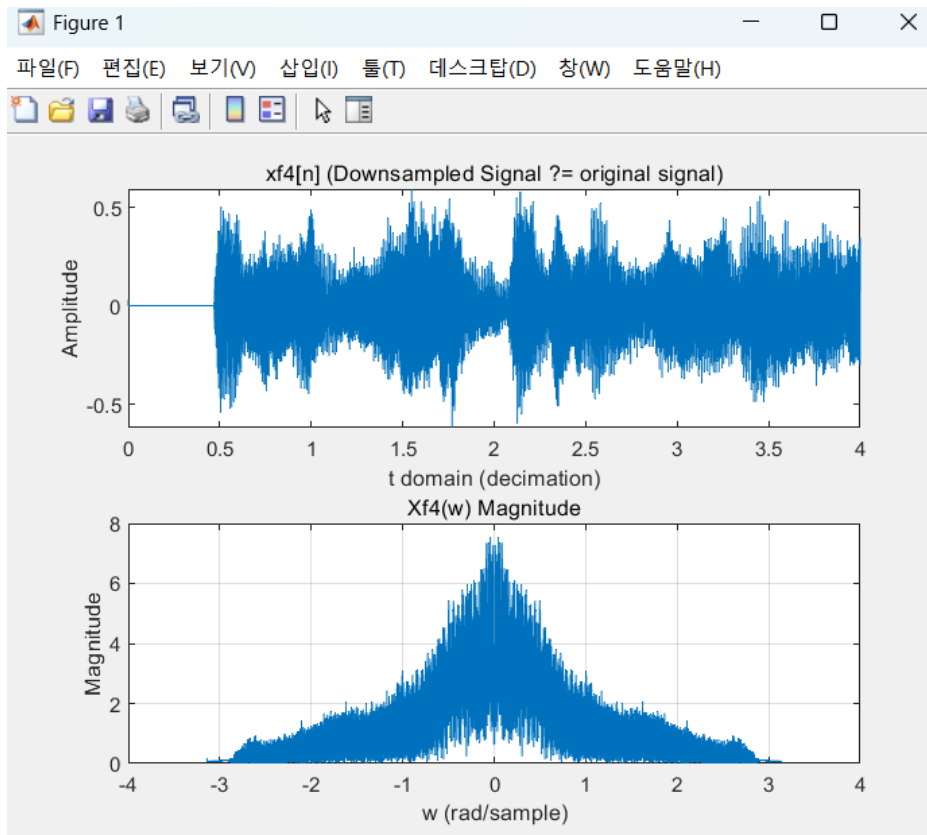
직접 소리를 들어보면, 고주파수 대역이 전부 사라졌기에 기존신호에서 저음역대로 들렸던 소리 위주로만 울리면서 들린다. 이 울리면서 들리는 현상이 원래 신호에서 저주파였던 신호들이 고주파로 이동하며 생기는 현상이라고 느꼈다.

6-2. Anti-Aliasing Filter 사용 안 하고 M=14로 Decimation



직접 소리를 들어보면, 저음역대 위주로만 들리는 것과 더불어, 이 또한 4번에서 다루었던 신호와 같이 잡음이 들린다. 이 잡음은 이 신호의 잡음은 4번의 잡음과는 다르게 약간 늘어지고 두꺼운 느낌을 주었다.

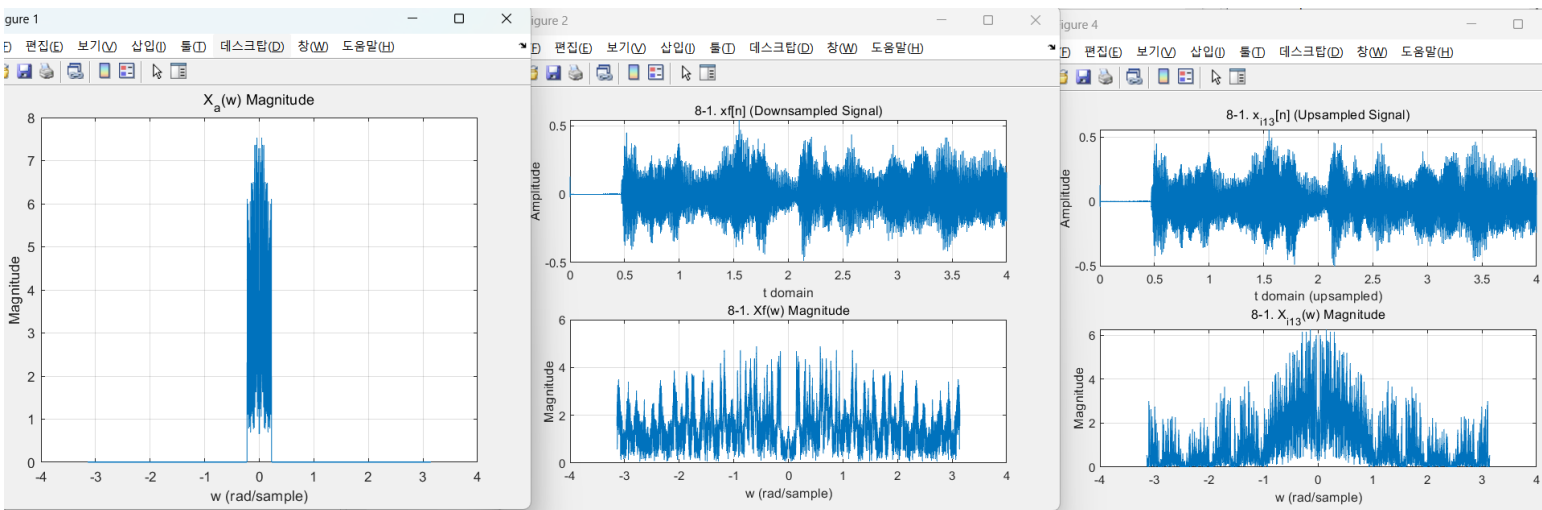
6-3. M=4로 설정 시 원신호로 복원되는가?



L=4를 통하여 upsampling된 n축 기준으로 index = 0, 4, 8, ...에 원신호의 정보들이 존재한다. 이를 다시 down sampling을 통하여 index 0, 4, 8에 있는 신호만 가져오므로 거의 같은 소리가 들리고, 파형도 거의 같다.

하지만, Interpolation을 할 때, non-ideal LPF를 사용했기에, 이 부분에 대한 소리의 변형이 존재했을 것이다.

6-4. Decimation 이후에 Interpolation시 발생하는 일



Decimation을 먼저 하게 되면 원 신호의 주파수 대역의 많은 부분을 Anti-Aliasing을 위하여 없애게 된다. 따라서, 신호의 많은 정보가 없어진다. Decimation은 본래 n -domain의 신호를 압축하여, 품질이 좋게 보이게 만드는 효과가 있지만, 이와 같이 많은 정보가 없어지면, 손실이 매우 크다. 그 후 interpolation을 하게 되면, 이미 많은 정보를 손실한 이미지의 n -domain 신호를 늘리기 위해 zero padding을 하고, LPF와 convolution하여 zero padding된 부분을 메꾼다.

결국, Interpolation을 먼저하고 Decimation을 한 신호와 Decimation을 먼저하고 Interpolation을 한 신호는 sampling rate는 같지만, 신호의 품질에 큰 차이가 발생한다.

따라서, Decimation을 먼저 하면 안 된다.

7. Appendix

```
clear; % 모든 변수 지우기
clc; % 명령창 지우기
close all; % 모든 figure 창 닫기

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 1-1 시작 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1-1. 신호 x[n]의 파형을 그리기 & sound

% x.wav 파일 읽기
[x, fs] = audioread('x.wav'); % x: 신호 데이터, fs: 샘플링 주파수

%===== 변수 setting =====%
n_domain = (0:length(x)-1); % n축 생성
t_domain = (0:length(x)-1)*(1/fs); % t축 생성

n = length(x); % 신호 길이
%=====

% x_zero(1:length(x)) = 0;

% % 파형 그리기
% figure;
% plot(n_domain, x);
% % plot(t_domain, x);
% xlabel('n domain');
% % xlabel('time domain (sec)');
% ylabel('Amplitude');
% title('Waveform of x[n]');
% grid on;

% 신호 청취
disp("1-1. playing x")
sound(x, fs);
pause(5); % 5초 대기 (신호 재생 시간)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 1-2 시작 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% FFT 계산

X = fft(x); % FFT 계산
X_shifted = fftshift(X); % FFT 결과를 중앙 정렬

%===== 변수 setting =====%
X_omega = linspace(-pi, pi, n); % w 축 생성 (주파수 범위:  $-\pi \sim \pi$ )
%=====

% Magnitude 계산 및 로그 스케일 변환
X_magnitude = abs(X_shifted); % Magnitude 계산
X_magnitude_log = log(1 + X_magnitude); % 로그 스케일

% % Magnitude 그래프 그리기
% figure;
% plot(X_omega, X_magnitude_log);
% xlabel('ω (rad/sample)');
% ylabel('Magnitude (log scale)');
% title('Magnitude Spectrum of X(ω) in Log Scale');
% grid on;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 1-3 시작 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1-3. Interpolation을 하기 전에, 이전 신호 x[n] 사이사이에 0을 삽입하는 과정 (Zero-padding)을 진행

L = 4; % Interpolation (L=4)

% Matrix size: 4*n 행, 1열
```

```

x_e = zeros(L*n, 1); % 0으로 채워진 신호 생성

% x_e의 1, 5, 9, ... 번째에 x[n]의 값을 넣음
x_e(1:L:end) = x; % 원래 신호를 4배 길이로 확장

%===== 변수 setting =====%
n_domain_e = (0:length(x_e)-1); % n_e축 생성
t_domain_e = (0:length(x_e)-1)/(L*fs); % t_e축 생성
%=====

% figure;
% % stem(n_domain_e, x_e);
% plot(t_domain_e, x_e);
% title('Waveform of x_e[n] (= x[n] with Zero-padding)');
% % xlabel('n domain (0 ~ 4n-1)');
% xlabel('time domain (sec)');
% ylabel('Amplitude');
% grid on;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 1-4 시작 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% disp("1-4. playing x_e")
% sound(x_e, fs*L); % sound signal (fs는 4배 증가)
pause(5); % 5초 대기 (신호 재생 시간)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 1-5 시작 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
X_e = fft(x_e); % FFT 계산
X_e_shifted = fftshift(X_e); % FFT 결과를 중앙 정렬
X_e_magnitude = abs(X_e_shifted); % Magnitude 계산
X_e_magnitude_log = log(1 + X_e_magnitude); % 로그 스케일 변환
upsample_omega = linspace(-pi, pi, L*n); % 주파수 벡터 생성

% figure;
% subplot(2, 1, 1); % 2행 1열의 첫 번째 subplot
% plot(X_omega, X_magnitude_log); % X(ω) Magnitude 그래프
% xlabel('ω (rad/sample)');
% ylabel('Magnitude (log scale)');
% title('Magnitude Spectrum of X(ω) in Log Scale');
% grid on;
% subplot(2, 1, 2); % 2행 1열의 두 번째 subplot
% plot(upsample_omega, X_e_magnitude_log); % X_e(ω) Magnitude 그래프
% xlabel('ω (rad/sample)');
% ylabel('Magnitude (log scale)');
% title('Magnitude Spectrum of X_e(ω) in Log Scale');
% grid on;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 2-1 시작 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2-1. H_r(w)를 알맞게 설계한 뒤, H_r(w)의 Magnitude 그래프를 log scale이 아닌 일반적인 linear scale로 그리고, 설명

upsample_n = L * length(x); % 신호 길이

upsample_omega = linspace(-pi, pi, upsample_n); % 주파수 벡터 생성

H_r = zeros(upsample_n, 1); % H_r(w) 필터 초기화

middle_index = floor(upsample_n / 2) + 1; % 필터의 중심 인덱스

% H_r의 middle에 L 값을 할당 (주파수 대역에 해당하는 부분)
start_index = middle_index - floor(upsample_n / L / 2);
end_index = middle_index + floor(upsample_n / L / 2);

H_r(start_index:end_index) = L;

% % Magnitude 그래프 그리기
% figure;
% plot(upsample_omega, H_r); % H_r(w) Magnitude 그래프

```

```

% xlabel('ω (rad/sample)');
% ylabel('Magnitude (linear scale)');
% title('Magnitude Spectrum of H_r(ω) in Linear Scale');
% grid on;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 2-2 시작 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% 2-2. 주파수 축에서 구현한 H(w)를 시간축에서의 h[n]로 변환한 뒤 아래와 같이 w5[n], w13[n]로 각각 windowing한 h5[n],
h13[n]을 구하세요.
% 그리고 둘의 파형을 그리세요.
% 단, h5[n], h13[n]의 모든 coefficient 합은 L이 되도록 normalize 해야 합니다.

%===== h_r[n] =====%
H_r_original = ifftshift(H_r); % H_r(w)의 0 rad/sec를 샘플 0에서 시작하도록 변환
h_r = ifftshift(ifft(H_r_original)); % H_r(w)를 n domain으로 변환 후 다시 중심을 이동
%=====

%===== w_5[n] =====%
w_5 = create_filter(5);
%=====

%===== w_13[n] =====%
w_13 = create_filter(13);
%=====

%===== h_r_5[n] =====%
h_r_5 = h_r(middle_index - 2 : middle_index + 2) .* w_5; % H_r(w)를 시간 영역으로 변환
%=====

%===== h_r_13[n] =====%
h_r_13 = h_r(middle_index - 6 : middle_index + 6) .* w_13; % H_r(w)를 시간 영역으로 변환
%=====

%==== Power 보정 : 모든 coefficient 합은 이 되도록 normalize ====%
% disp(sum(h_r))
h_r_5 = h_r_5 * L / sum(h_r_5);
h_r_13 = h_r_13 * L / sum(h_r_13);
% power는 이렇게 아닌가?? 원래??
% h_r_5 = h_r_5 * sqrt(sum(abs(h_r).^2) / sum(abs(h_r_5).^2));
% h_r_13 = h_r_13 * sqrt(sum(abs(h_r).^2) / sum(abs(h_r_13).^2));
%=====

%===== 변수 setting =====%

n_5_domain = linspace(-2, 2, length(h_r_5)); % h_r_5[n]의 n 축
n_13_domain = linspace(-6, 6, length(h_r_13)); % h_r_13[n]의 n 축
upsample_n_domain = linspace(-upsample_n / 2, upsample_n / 2, length(h_r)); % 전체 upsample_n 축
%=====

% figure;
% stem(upsample_n_domain, h_r); % h_r[n] 파형 그래프
% xlabel('upsameple n');
% ylabel('Amplitude');
% title('Waveform of h_r[n]');
% grid on;
%
% figure;
% subplot(2, 1, 1);
% stem(n_5_domain, h_r_5); % h_r_5[n] 파형 그래프
% xlabel('n_5 domain');
% ylabel('Amplitude');
% title('Waveform of h_r_5[n]');
% grid on;
%
% subplot(2, 1, 2);
% stem(n_13_domain, h_r_13); % h_r_13[n] 파형 그래프
% xlabel('n_1_3 domain');
% ylabel('Amplitude');
% title('Waveform of h_r_1_3[n]');
% grid on;
%=====

```



```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 2-3 시작 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2-3. h5[n], h13[n]의 Frequency Response
% H5(w), H13(w)의 Magnitude를
% log scale이 아닌 일반적인 linear scale로 그리고, H(w)와 비교하세요.

%== H_r_5의 주파수 응답 및 H_r_13의 주파수 응답을 계산 ==%
H_r_2 = fftshift(fft(h_r, length(H_r)));
H_r_5 = fftshift(fft(h_r_5, length(H_r)));
H_r_13 = fftshift(fft(h_r_13, length(H_r)));
% H_r_5 = fftshift(fft(h_r_5));
% H_r_13 = fftshift(fft(h_r_13));
% H_r_5_omega = linspace(-pi, pi, length(H_r_5));
% H_r_13_omega = linspace(-pi, pi, length(H_r_13));
%=====

% figure;
% subplot(3, 1, 1);
% plot(upsample_omega, abs(H_r_2)); % H_r(w) Magnitude 그래프
% xlabel('ω (rad/sample)');
% ylabel('Magnitude (linear scale)');
% title('Magnitude Spectrum of H_r(ω) in Linear Scale');
% grid on;
%

% subplot(3, 1, 2);
% plot(upsample_omega, abs(H_r_5)); % H_r(w) Magnitude 그래프
% % stem(H_r_5_omega, abs(H_r_5)); % H_r(w) Magnitude 그래프
% xlabel('ω (rad/sample)');
% ylabel('Magnitude (linear scale)');
% title('Magnitude Spectrum of H_r_5(ω) in Linear Scale');
% grid on;
%

% subplot(3, 1, 3);
% plot(upsample_omega, abs(H_r_13)); % H_r(w) Magnitude 그래프
% % stem(H_r_13_omega, abs(H_r_13)); % H_r(w) Magnitude 그래프
% xlabel('ω (rad/sample)');
% ylabel('Magnitude (linear scale)');
% title('Magnitude Spectrum of H_r_13(ω) in Linear Scale');
% grid on;
%=====

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 3-1 시작 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3-1. 신호 xe[n]을 Reconstruction Filter hr5[n], hr13[n]을 이용하여
% time domain에서 filtering을 진행한 신호 x5[n], x13[n]을 구하세요. 그리고 둘의 파형을 그리세요.

%===== x_i_5 및 x_i_13을 convolution 계산 =====%
x_i_5 = conv(x_e, h_r_5, 'same'); % x[n]을 h_r_5[n]으로 convolution
x_i_13 = conv(x_e, h_r_13, 'same'); % x[n]을 h_r_13[n]으로 convolution
%=====

% disp(length(x_i_5));
% disp(length(x_i_13));

%===== n축 생성 =====%
% 1-3번에서 정의함
%=====

% figure;
% subplot(3, 1, 1);
% % plot(n_domain_e, x_e); % x_e[n] plot
% stem(t_domain_e, x_e); % x_e[n] plot
% % xlabel('n domain (0 ~ 4n-1)');
% xlabel('time domain (sec)');
% ylabel('Amplitude');
% title('Waveform of x_e[n]');
% grid on;

% subplot(3, 1, 2);
% % plot(n_domain_e, x_i_5); % x_i_5[n] plot

```

[illegible]

```

% M=14인 경우 xp[n]을 구하고 파형을 그리기
x_p = zeros(size(x_i_13)); % xp[n] 초기화
M = 14; % M 값 설정
x_p(1:M:end) = x_i_13(1:M:end); % M 간격으로 x_i_13[n]의 값을 할당

downsample_n = length(x_p); % xp[n]의 길이

%===== n_down 축 생성 =====%
downsample_n_domain = (0:downsample_n-1);
downsample_t_domain = (0:downsample_n-1)/(L*fs);
%=====

% figure;
% stem(downsample_n_domain, x_p); % xp[n] plot
% plot(downsample_t_domain, x_p); % xp[n] plot
% xlabel('n domain');
% ylabel('t domain');
% ylabel('Amplitude');
% title('xp[n] with M=14');

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 4-2 시작 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4-2. xp[n]의 Spectrum Xp(w)의 magnitude 그래프를 그리고 설명하세요.

%===== Xp(w) 계산 =====%
% Xp(w) 계산을 위한 FFT 수행
Xp = fft(x_p); % xp[n]의 FFT
%=====

% figure;
% plot(upsample_omega, abs(Xp)); % xp[n]의 파형
% xlabel('w (rad/sample)');
% ylabel('Magnitude');
% title('X_p(w)');
% grid on;
%=====

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 4-3 시작 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4-3. xp[n]을 청취하고, 원래 신호 x13[n]과 어떤 차이가 있는지 서술하세요.
% xp[n] = x_i_13[n] for n = Mk
% 0 otherwise

%===== sound x_i_13 및 x_p =====%
% disp("4-3. playing x_i_13")
% sound(x_i_13, L*fs); % x_i_13[n] 재생
% pause(5); % 5초 대기

% disp("4-3. playing x_p")
% sound(x_p, L*fs); % xp[n] 재생
% pause(5); % 5초 대기

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 4-4 시작 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4-4. Down-sampling을 하기 위해 문제 3에서 얻은 신호 xp[n]에 대해,
% 아래의 식을 사용하여 xd[n]의 파형을 그리세요. (M=14)

%===== xd[n] =====%
x_d = zeros(ceil(size(x_p)/M)); % xd[n] 초기화
x_d(1:1:end) = x_p(1:M:end); % M 간격으로 x_p[n]의 값을 할당

% disp(length(x_d));

decimation_n_domain = (0:length(x_d)-1); % n_downsample 축 생성
decimation_t_domain = (0:length(x_d)-1)/(L*fs/M); % n_downsample 축 생성
%=====

% figure;
% subplot(2, 1, 1);

```

```

% % plot(downsample_n_domain, x_p); % xp[n] 파형
% plot(downsample_t_domain, x_p); % xp[n] 파형
% % xlabel('n domain');
% xlabel('t domain');
% ylabel('Amplitude');
% title('xp[n]');
% grid on;

% subplot(2, 1, 2);
% % plot(decimation_n_domain, x_d); % xd[n] 파형
% plot(decimation_t_domain, x_d); % xd[n] 파형
% % xlabel('n downsampling domain');
% xlabel('t domain');
% ylabel('Amplitude');
% title('xd[n]');
% grid on;
%=====

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 4-5 시작 %%%%%%%%%
% 4-5. xd[n]의 Spectrum Xd(w)의 magnitude 그래프를 그리세요.

%===== Xd(w) =====
Xd = fftshift(fft(x_d)); % xd[n]의 FFT 계산
%=====

%===== Xd(w) magnitude plot =====
downsample_omega = linspace(-pi, pi, length(Xd)); % w 축 생성
%=====

% figure;
% plot(downsample_omega, abs(Xd)); % Xd(w)의 magnitude plot
% xlabel('w (rad/sample)');
% ylabel('Magnitude');
% title('Xd(w)');
% grid on;
%=====

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 4-6 시작 %%%%%%%%%
%===== sound(x_d, L*fs/M) 및 sound(x_p, L*fs) =====
% disp("4-6. playing xd")
% sound(x_d, L*fs/M); % xd[n]을 재생
% pause(5); % 5초 대기

% disp("4-6. playing xp")
% sound(x_p, L*fs); % xp[n]을 재생
% pause(5);
%=====

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 5-1 시작 %%%%%%%%%
%===== Ha(w) 디자인 =====
H_a = zeros(upsample_n, 1); % H_r(w) 필터 초기화

middle_index = floor(upsample_n / 2) + 1; % 필터의 중심 인덱스

% H_a의 중앙 부분에 L 값을 할당 (주파수 대역에 해당하는 부분)
start_index = middle_index - floor(upsample_n / M / 2);
end_index = middle_index + floor(upsample_n / M / 2);

% disp(middle_index);
% disp(start_index);
% disp(end_index);

H_a(start_index:end_index) = 1; % 중앙 대역에 L 값 할당
%=====
%
% figure;
% plot(upsample_omega, H_a); % Ha(w) magnitude plot
% xlabel('w (rad/sample)');
% ylabel('Magnitude');

```

```

% title('Ha(w) Magnitude Plot');
% grid on;
%=====

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 5-2 시작 %%%%%%%%%%%%%
X_a = H_a .* X_i_13;           % Xa(w) = multiply(Ha(w), X_i_13(w))
x_a = (ifft(ifftshift(X_a)));   % xa[n] = IDFT(Xa(w))

% n_a = (0:upsample_n-1);       % n_a 축 생성

% figure;
% % plot(n_domain_e, x_a);
% plot(t_domain_e, x_a);
% % xlabel('n domain (upsampled)');
% xlabel('t domain');
% ylabel('Amplitude');
% title('x_a[n]');
% grid on;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 5-3 시작 %%%%%%%%%%%%%
% X_a = H_a .* X_i_13;           % Xa(w) = multiply(Ha(w), X_i_13(w))

% figure;
% subplot(3,1,1);
% plot(upsample_omega, log(1+abs(X_a)));           % Ha(w) magnitude plot
% % plot(upsample_omega, abs(X_a));               % Ha(w) magnitude plot
% xlabel('w (rad/sample)');
% ylabel('Magnitude (log scale)');
% title('X_a(w) magnitude plot');
% grid on;
%
% subplot(3,1,2);
% plot(X_omega, X_magnitude_log);                 % X(w) magnitude plot
% % plot(X_omega, X_magnitude);                  % X(w) magnitude plot
% xlabel('w (rad/sample)');
% ylabel('Magnitude (log scale)');
% title('X(w) magnitude plot');
% grid on;
%
% subplot(3,1,3);
% plot(upsample_omega, log(1+abs(X_i_13)));         % X_i_13(w) magnitude plot
% % plot(upsample_omega, abs(X_i_13));            % X_i_13(w) magnitude plot
% xlabel('w (rad/sample)');
% ylabel('Magnitude (log scale)');
% title('X_i_13(w) magnitude plot');
% grid on;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 5-4 시작 %%%%%%%%%%%%%
x_f = zeros(ceil(size(x_a)/M));           % xf[n] 초기화
x_f(1:1:end) = x_a(1:M:end);             % M 간격으로 x_a[n]의 값을 할당
%=====

% figure;
% % plot(decimation_n_domain, x_f);               % xf[n] plot
% plot(decimation_t_domain, x_f);               % xf[n]의 t-domain plot
% % xlabel('n domain (decimation)');
% xlabel('t domain (sec)');
% ylabel('Amplitude');
% title('xf[n] (Downsampled Signal)');
% grid on;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 5-5 시작 %%%%%%%%%%%%%
X_f = fftshift(fft(x_f));                 % xf[n]의 FFT 계산
magnitude_Xf = abs(X_f);                  % xf[n]의 magnitude 계산
log_magnitude_Xf = log(1 + magnitude_Xf);  % log scale로 변환
%=====

% figure;
% subplot(2, 1, 1);
% plot(downsample_omega, log(1+abs(Xd)));         % Xd(w) magnitude plot
% xlabel('w (rad/sample)');
% ylabel('Magnitude');
% title('Xd(w) Magnitude');
% grid on;

```

```

% subplot(2, 1, 2);
% plot(downsample_omega, log_magnitude_Xf);      % Xf(w) magnitude plot
% xlabel('w (rad/sample)');
% ylabel('Magnitude');
% title('Xf(w) Magnitude');
% grid on;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 5-6 시작 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
disp("5-6. playing xd[n]");
sound(x_d, L*fs/M);                          % xd[n] 청취
pause(5);                                     % 5초 대기

disp("5-6. playing xf[n]");
sound(x_f, L*fs/M);                          % xf[n] 청취
pause(5);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Discussion 시작 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 6-1 시작 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 만약 Interpolation을 선행하지 않고 decimation 진행

% Anti-Alias Filter 사용하여 design
%===== Ha2(w) 디자인 =====%
H_a2 = zeros(n, 1);                          % H_r(w) 필터 초기화

middle_index = floor(n / 2) + 1;              % 필터의 중심 인덱스

% H_a의 중앙 부분에 L 값을 할당 (주파수 대역에 해당하는 부분)
start_index = middle_index - floor(n / M / 2);
end_index = middle_index + floor(n / M / 2);

% disp(middle_index);
% disp(start_index);
% disp(end_index);

H_a2(start_index:end_index) = 1;              % 중앙 대역에 L 값 할당
%=====

%===== X_a2(w) 계산 =====%
X_a2 = H_a2 .* fftshift(X);                  % Xa(w) = multiply(Ha(w), X_i_13(w))
x_a2 = (ifft(ifftshift(X_a2)));              % xa[n] = IDFT(Xa(w))

% figure;
% plot(X_omega, log(1+abs(fftshift(X))));
% xlabel('w (rad/sample)');
% ylabel('Magnitude');
% title('X(w) magnitude plot');
% grid on;

% figure;
% subplot(2, 1, 1);
% % plot(n_domain, x_a2);
% plot(t_domain, x_a2);
% xlabel('t domain');
% ylabel('Amplitude');
% title('x_a2[n] (After Decimation)');
% grid on;

% subplot(2, 1, 2);
% plot(X_omega, log(1+abs(X_a2)));              % Ha(w) magnitude plot
% xlabel('w (rad/sample)');
% ylabel('Magnitude');
% title('X_a2(w) magnitude plot');
% grid on;
%=====

%===== x_f2[n] 계산 =====%
x_f2 = zeros(ceil(size(x_a2)/M));            % xf[n] 초기화 ***** why ceil??

```

```

x_f2(1:1:end) = x_a2(1:M:end); % M 간격으로 x_a[n]의 값을 할당
=====

%===== x_f2(w) 계산 =====%
X_f2 = fftshift(fft(x_f2)); % xf[n]의 FFT 계산
magnitude_Xf2 = abs(X_f2); % xf[n]의 magnitude 계산
log_magnitude_Xf2 = log(1 + magnitude_Xf2); % log scale로 변환
=====

%===== x_f2[n] 및 X_f2(w)의 축 생성 =====%
ninterpolation_n_domain = (0:length(x_f2)-1); % n_f2 축 생성
ninterpolation_t_domain = (0:length(x_f2)-1)/(fs/M); % t_f2 축 생성
nointerpolation_omega = linspace(-pi, pi, length(X_f2)); % omega 축 생성
=====

% figure;
% subplot(2, 1, 1);
% % plot(ninterpolation_n_domain, x_f2); % xf2[n]의 n-domain plot
% plot(ninterpolation_t_domain, x_f2); % xf2[n]의 t-domain plot
% xlabel('t domain (decimation)');
% ylabel('Amplitude');
% title('xf2[n] (Downsampled Signal)');
% grid on;

% subplot(2, 1, 2);
% plot(ninterpolation_omega, log_magnitude_Xf2); % Xf(w) magnitude plot
% xlabel('w (rad/sample)');
% ylabel('Magnitude');
% title('X_f_2(w) Magnitude');
% grid on;
=====

% disp("6-1. playing xf2[n]");
% sound(x_f2, fs/M); % xf2[n] 청취fs/M
% pause(5); % 5초 대기

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 6-2 시작 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 만약 Interpolation을 선행하지 않고 decimation 진행

% Anti-Alias Filter 사용 안하고 design
%===== x_p3(n) 디자인 =====%
x_p3 = zeros(size(x));
x_p3(1:M:end) = x(1:M:end); % x[n]을 M 간격으로 할당
=====

%===== X_p3(w) 계산 =====%
X_p3 = fftshift(fft(x_p3)); % Xp(w) 계산
=====

% figure;
% subplot(2, 1, 1);
% % plot(n_domain, x_p3);
% plot(t_domain, x_p3);
% xlabel('t domain');
% ylabel('Amplitude');
% title('x_p_3[n] (Downsampled Signal)');
% grid on;

% subplot(2, 1, 2);
% plot(X_omega, log(1+abs(X_p3))); % Xp(w) magnitude plot
% xlabel('w (rad/sample)');
% ylabel('Magnitude');
% title('X_p_3(w) magnitude plot');
% grid on;

%===== x_f3[n] 계산 =====%

```

```

x_f3 = zeros(ceil(size(x)/M)); % xf[n] 초기화 ***** ceil 안하면 오류남 왜지?
x_f3(1:1:end) = x(1:M:end); % M 간격으로 x_a[n]의 값을 할당
=====

%===== x_f3(w) 계산 =====%
X_f3 = fftshift(fft(x_f3)); % xf[n]의 FFT 계산
magnitude_Xf3 = abs(X_f3); % xf[n]의 magnitude 계산
log_magnitude_Xf3 = log(1 + magnitude_Xf3); % log scale로 변환
=====

%===== x_f3[n] 및 X_f3(w)의 축 생성 =====%
ninterpolation_n_domain = (0:length(x_f3)-1); % n_f2 축 생성
ninterpolation_t_domain = (0:length(x_f3)-1)/(fs/M); % t_f2 축 생성
noiterpolation_omega = linspace(-pi, pi, length(X_f3)); % 주파수 벡터 생성
=====

% figure;
% subplot(2, 1, 1);
% % plot(ninterpolation_n_domain, x_f3);
% plot(ninterpolation_t_domain, x_f3);
% xlabel('t domain (decimation)');
% ylabel('Amplitude');
% title('x_f_3[n] (Downsampled Signal)');
% grid on;

% subplot(2, 1, 2);
% plot(ninterpolation_omega, log_magnitude_Xf3); % Xf(w) magnitude plot
% xlabel('w (rad/sample)');
% ylabel('Magnitude');
% title('X_f_3(w) Magnitude');
% grid on;
=====

% disp("6-2. playing xf3[n]");
% sound(x_f3, fs/M); % xf2[n] 청취fs/M
% pause(5); % 5초 대기

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 7-1 시작 %%%%%%%%%
% L=4, M=4인 경우 원신호로 되돌아가는지

M2 = 4;

%===== Ha(w) 디자인 =====%
H_a4 = zeros(upsample_n, 1); % H_r(w) 필터 초기화

middle_index = floor(upsample_n / 2) + 1; % 필터의 중심 인덱스

% H_a의 중앙 부분에 L 값을 할당 (주파수 대역에 해당하는 부분)
start_index = middle_index - floor(upsample_n / M2 / 2);
end_index = middle_index + floor(upsample_n / M2 / 2);

H_a4(start_index:end_index) = 1; % 중앙 대역에 L 값 할당
=====
%
% figure;
% plot(upsample_omega, H_a4); % Ha(w) magnitude plot
% xlabel('w (rad/sample)');
% ylabel('Magnitude');
% title('Ha(w) Magnitude Plot');
% grid on;
=====

X_a = H_a4 .* X_i_13; % Xa(w) = multiply(Ha(w), X_i_13(w))
x_a = (ifft(ifftshift(X_a))); % xa[n] = IDFT(Xa(w))

x_f4 = zeros(ceil(size(x_a)/M2)); % xf[n] 초기화
x_f4(1:1:end) = x_a(1:M2:end); % M 간격으로 x_a[n]의 값을 할당
=====

```



```

%=====변수 setting=====
x_f4_n_domain = (0:length(x_f4)-1);
x_f4_t_domain = (0:length(x_f4)-1)/fs;
X_f4_omega = linspace(-pi, pi, length(x_f4));

X_f4 = fftshift(fft(x_f4)); % xf[n]의 FFT 계산
magnitude_Xf4 = abs(X_f4); % xf[n]의 magnitude 계산
log_magnitude_Xf4 = log(1 + magnitude_Xf4); % log scale로 변환

% figure;
% subplot(2, 1, 1);
% plot(x_f4_t_domain, x_f4); % xf[n]의 t-domain plot
% xlabel('t domain (decimation)');
% ylabel('Amplitude');
% title('xf4[n] (Downsampled Signal ?= original signal)');
%
% subplot(2, 1, 2);
% plot(X_f4_omega, log_magnitude_Xf4); % Xf(w) magnitude plot
% xlabel('w (rad/sample)');
% ylabel('Magnitude');
% title('Xf4(w) Magnitude');
% grid on;

%===== 소리 청취 =====
% disp("7-1.playing xf4[n]");
% sound(x_f4, L*fs/M2); % xf[n] 청취
% pause(5);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% 8-1 시작 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Decimation 먼저 하고, Interpolation 진행

clear; % 모든 변수 지우기
clc; % 명령창 지우기
close all; % 모든 figure 창 닫기

% x.wav 파일 읽기
[x, fs] = audioread('x.wav'); % x: 신호 데이터, fs: 샘플링 주파수

%===== X(w) 구하기 =====
X = fft(x); % FFT 계산
X_shifted = fftshift(X); % FFT 결과를 중앙 정렬

%===== 변수 setting =====
n = length(x); % x[n]의 길이
n_domain = (0:length(x)-1); % n축 생성
t_domain = (0:length(x)-1)*(1/fs); % t축 생성
X_omega = linspace(-pi, pi, length(X));
%=====

%===== Decimation =====
M = 14;
%===== Ha(w) 디자인 =====
H_a = zeros(n, 1); % H_r(w) 필터 초기화

middle_index = floor(n / 2) + 1; % 필터의 중심 인덱스

% H_a의 중앙 부분에 L 값을 할당 (주파수 대역에 해당하는 부분)
start_index = middle_index - floor(n / M / 2);
end_index = middle_index + floor(n / M / 2);

H_a(start_index:end_index) = 1; % 중앙 대역에 L 값 할당

%===== X_a에 LPF 씌우고 IFFT =====
X_a = H_a .* X_shifted; % Xa(w) = multiply(Ha(w), X(w))
x_a = (ifft(ifftshift(X_a))); % xa[n] = IDFT(Xa(w))

```

```

%=====
figure;
plot(X_omega, log(1+abs(X_a))); % X(w) magnitude plot
xlabel('w (rad/sample)');
ylabel('Magnitude');
title('X_a(w) Magnitude');
grid on;

%===== Down Sampling 된 x_f 생성 =====
x_f = zeros(ceil(size(x_a)/M)); % x_f[n] 초기화
x_f(1:1:end) = x_a(1:M:end); % M 간격으로 x_a[n]의 값을 할당

downsample_n = length(x_f); % x_f[n]의 길이

%===== X_f(w) 생성 =====
X_f = fft(x_f); % x_f[n]의 FFT 계산
X_f_shifted = fftshift(X_f); % FFT 결과를 중앙 정렬

%===== n_down 축 및 w 축 생성 =====
downsample_n_domain = (0:downsample_n-1);
downsample_t_domain = (0:downsample_n-1)/(fs/M);
downsample_omega = linspace(-pi, pi, length(x_f));
%=====

figure;
subplot(2, 1, 1);
plot(downsample_t_domain, x_f); % x_f[n] plot
xlabel('t domain');
ylabel('Amplitude');
title('8-1. x_f[n] (Downsampled Signal)');
grid on;

subplot(2, 1, 2);
plot(downsample_omega, log(1+abs(X_f_shifted))); % X_d(w) magnitude plot
xlabel('w (rad/sample)');
ylabel('Magnitude');
title('8-1. X_f(w) Magnitude');
grid on;

%===== Interpolation =====
L = 4;

%===== Zero Padding x_e[n] 생성 =====
upsample_n = downsample_n * L; % upsampled x[n]의 길이

x_e = zeros(upsample_n, 1); % x_e[n] 초기화
x_e(1:L:end) = x_f; % L 간격으로 x_f[n]의 값을 할당

%===== X_e(w) 생성 =====
X_e = fft(x_e); % x_e[n]의 FFT 계산
X_e_shifted = fftshift(X_e); % FFT 결과를 중앙 정렬

%===== gain L인 reconstruct LPF 생성 =====
H_r = zeros(upsample_n, 1); % H_r(w) 필터 초기화
middle_index = floor(upsample_n / 2) + 1; % 필터의 중심 인덱스
start_index = middle_index - floor(upsample_n / L / 2); % 시작 인덱스
end_index = middle_index + floor(upsample_n / L / 2); % 끝 인덱스
H_r(start_index:end_index) = L; % 중앙 대역에 L 값 할당

%===== H_r(w)를 IFFT =====
H_r_original = ifftshift(H_r);
h_r = ifftshift(ifft(H_r_original)); % h_r[n] = IDFT(H_r(w))
h_r_13 = h_r_13 * L / sum(h_r_13); % 정규화

%===== 13point windowing =====
w_13 = create_filter(13); % 13point windowing
h_r_13 = h_r(middle_index - 6 : middle_index + 6) .* w_13; % 13point windowing 적용

```

```

%=====

n_13_domain = linspace(-6, 6, length(h_r_13));
figure;
stem(n_13_domain, h_r_13); % h_r[n] plot
xlabel('n domain (13 point windowing)');
ylabel('Amplitude');
title('h_r[n] (13 point windowing)');
grid on;

%===== convolution [x_e * h_r_13] =====%
x_i_13 = conv(x_e, h_r_13, 'same'); % x[n]을 h_r_13[n]으로 convolution

%===== x_i_13의 FFT =====%
X_i_13 = fft(x_i_13); % x_i_13[n]의 FFT 계산
X_i_13_shifted = fftshift(X_i_13); % FFT 결과를 중앙 정렬

%=====interpolation 신호 domain 생성 =====%
upsample_n_domain = (0:upsample_n-1);
upsample_t_domain = (0:upsample_n-1)/(L*fs/M);
upsample_omega = linspace(-pi, pi, upsample_n); % 주파수 벡터 생성
%=====
figure;
subplot(2, 1, 1);
plot(upsample_t_domain, x_i_13); % x_i_13[n] plot
xlabel('t domain (upsampled)');
ylabel('Amplitude');
title('8-1. x_i_13[n] (Upsampled Signal)');
grid on;

subplot(2, 1, 2);
plot(upsample_omega, log(1+abs(X_i_13_shifted)));
xlabel('w (rad/sample)');
ylabel('Magnitude');
title('8-1. X_i_13(w) Magnitude');
grid on;

%===== 소리 청취 =====%
disp("8-1. playing x_i_13[n]");
sound(x_i_13, L*fs/M);
pause(5);
%=====

%=====

% window making function
function w = create_filter(N)
    w = ones(N, 1); % 필터 크기만큼 1으로 초기화
end

```